

RAPPORT SAE ROBOT

2023-24
BUT1 S2 GEII
Université Côte d'Azur



Groupe 2 – Poste 5-6

CASA Cost

FRANCES Quentin

CIAMPI Corentin

BARROIS Victor

Enseignants de projet :

LE-BASTARD Cédric

SALVAT Jean-Louis

Table des matières

Introduction.....	2
I) Le robot du semestre 2 et ses nouveautés par rapport au semestre 1.	3
1. Les nouvelles épreuves	3
2. Les nouveaux objectifs	4
3. La carte hacheur 4 quadrants	5
II) La carte mezzanine	8
1. Comment créer les tensions nécessaires ?	8
a. Créer du -5V.	8
b. Créer du 3V3	8
2. Le traitement du signal	9
a. Capter le signal infra-rouge	9
b. Supprimer la constante continue	10
c. Amplifier le signal	11
d. Isoler le 5kHz	12
e. Récupérer l'enveloppe du signal	14
f. Rendre le signal binaire	15
3. Les capteurs ultra-sons.....	16
4. L'écran LCD.....	18
5. Liste des composants et répartition du travail	20
III) Les compléments.....	22
1. Le module Grove MP3	22
2. Le module Bluetooth	27
IV) Schéma de câblage	29
V) La programmation du robot	31
1. Les encodeurs	31
2. L'accéléromètre	32
3. Le carré	34
4. Le suivi de ligne.....	35
5. Eviter un obstacle	36
6. Détecter la balise et s'en rapprocher.....	37
VI) L'organisation dans l'équipe	39
Conclusion	41
Table des figures.....	42
Annexes	43

Introduction

Durant le semestre 2 de la formation à l'IUT dans le département GEII (Génie électrique et informatique industrielle), nous avons pour la seconde fois participé à un module SAé (situation d'apprentissage et d'évaluation).

Cette SAé du semestre 2 fait écho à celle du semestre 1 car nous reprenons la base du robot que nous avons conçu lors du semestre précédent afin de lui apporter des améliorations dans le but de remplir un nouveau cahier des charges.

A la différence du semestre 1, il n'y a pas de concours, mais des épreuves d'évaluations ont eu lieu le lundi 3 juin 2024. Ces épreuves étaient connues à l'avance, et ont donc été préparées par groupe de 4. Un groupe constitué pour notre part de BARROIS Victor, CIAMPI Corentin, COST Casa et FRANCES Quentin.

Nous avons dû à nouveau nous organiser afin d'être le plus efficace dans les nombreuses tâches que nous ont été demandés. Ce semestre apporte un aspect plus technique de par la conception d'une nouvelle carte, avec beaucoup d'électronique, que nous avons dû étudier et concevoir de A à Z, ce qui fut très enrichissant.

Ce rapport de projet vous présente l'avancement final de notre travail, à la date des épreuves évaluatives.

Vous retrouverez toutes les différentes étapes qui nous ont permis d'obtenir un robot fini. Après une présentation du nouveau cahier des charges, ce rapport sera principalement consacré à l'aspect électronique puis à l'aspect informatique.

I) Le robot du semestre 2 et ses nouveautés par rapport au semestre 1.

1. Les nouvelles épreuves

Avant de vous présenter les nouvelles épreuves et changements par rapport au cahier des charges du premier semestre, il est important de notifier que le deuxième semestre a pour but de concevoir de nouvelles cartes, afin que le robot possède de nouvelles capacités.

Forcément, de nouvelles épreuves sont créées pour utiliser les nouveaux mécanismes du robot mais on retrouve quelques épreuves du premier semestre dans les épreuves du S2. Comme pour le concours du S1, il y a des épreuves obligatoires et d'autres, facultatives. Durant chaque épreuve, les groupes sont autonomes lors du démarrage du robot et ils sont responsables en cas d'échec.

Passons maintenant à la présentation des épreuves : nous retrouvons cette fois-ci 10 épreuves au lieu de 4 au S1, ce qui est considérable. En effet, qui dit plus d'épreuves dit aussi plus de travail dans la conception des nouvelles cartes, et potentiellement plus de difficultés dans le code. Cependant, tous les enseignements suivis durant l'année, sont là pour nous apporter toutes les clés pour réussir. Au premier semestre, les épreuves étaient le suivi de ligne, les confettis, le carré et le cercle alors qu'au deuxième semestre nous retrouvons le suivi de ligne, le carré, le passage dans un labyrinthe, la détection de balises avec différentes fréquences, l'affichage d'un message sur un écran LCD, le lancement d'une musique ou bruitage sur un MP3 intégré, la télécommande en Bluetooth du robot et le démarrage du robot avec l'accéléromètre.

Voici donc un tableau récapitulatif, avec en gras les épreuves similaires :

Semestre 1	Semestre 2
Suivi de ligne	Suivi de ligne + LCD + MP3
Confettis	Démarrer le robot à l'accéléromètre
Carré	Suivi de ligne + s'arrêter devant un obstacle
Cercles	Carré avec les encodeurs
	Eviter un obstacle
	Sortir d'un labyrinthe
	Détecter une balise 5kHz
	Détecter une balise 5kHz + contourner un obstacle
	Détecter une balise 500Hz

Épreuves obligatoires : suivre une ligne en affichant des messages sur le LCD et en jouant de la musique, suivre une ligne en s'arrêtant devant un obstacle, sortir d'un labyrinthe, détecter une balise 5kHz, s'en rapprocher.

Épreuves facultatives : démarrer le robot avec l'accéléromètre, faire un carré en utilisant les encodeurs, éviter un obstacle en le contournant, détecter une balise 5kHz, s'en rapprocher en évitant un obstacle, télécommander en Bluetooth le robot, détecter une balise 500Hz, s'en rapprocher.

Explications des épreuves :

L'épreuve du suivi de ligne consiste à suivre une ligne noire avec plusieurs virages et le robot doit s'arrêter à la fin de la piste, mais en plus de cela, le robot doit afficher des messages sur un écran LCD (par exemple : "Le robot est en train de rouler") et faire de la musique (au choix) avec un MP3 tout le long du trajet.

Le démarrage du robot avec l'accéléromètre consiste à faire démarrer le robot en bougeant le robot sur un ou plusieurs axes (x, y ou z).

L'épreuve du carré consiste à réaliser le carré le plus « parfait » possible en utilisant les encodeurs des moteurs du robot afin de pouvoir faire des rotations très propres.

L'épreuve de la détection de balise consiste à reconnaître la balise selon une fréquence déterminée par l'épreuve et s'en approcher. Tout cela pourra être effectué par une carte placée sur le robot.

L'épreuve du Bluetooth consiste à piloter le robot à distance grâce à une application sur un téléphone IOS ou Android.

2. Les nouveaux objectifs

Alors qu'au premier semestre, les objectifs principaux étaient de réaliser les cartes capteurs, la carte hacheur, la carte testeur de batterie et la carte commande, deux nouvelles cartes s'ajoutent au programme avec la carte mezzanine et la mini carte d'adaptation pour la carte Grove MP3 qui permet d'ajouter de la musique ou des bruitages.

De plus, la carte mezzanine, qui vient compléter les capacités du robot en se connectant à la carte commande, remplace la carte IHM du 1^{er} semestre. Cette dernière est essentielle, car elle permet de réaliser les épreuves de la balise, du labyrinthe et de l'écran LCD I2C. Le fonctionnement des deux nouvelles cartes sera détaillé par la suite. Il faut également faire une communication Bluetooth, améliorer les

programmes du robot pour s'adapter aux nouvelles épreuves et mettre au propre le robot du S1 (câblage, fixation des cartes, décoration du robot).

Étant donné que nous sommes en SAé Robot, plusieurs matières fonds écho à celle-ci, notamment l'électronique. En effet, les nouveautés du S2 sont les techniques de filtrages des signaux et c'est cette partie qui prend le plus de temps à réaliser à cause des montages complexes. Les filtres utilisés pour le robot sont des filtres actifs, c'est-à-dire que l'on réalise des filtres passe-bas / passe-haut ou passe-bande et l'on ajoute un AOP que nous allons alimenter et placer en cascade comme vus en cours d'électronique pour filtrer et amplifier le signal par exemple. Dans un premier temps, le filtre est étudié théoriquement dans le polycopié de SAé Robot et en simulation sur des logiciels tels que FilterPro et LTSpice pour déterminer les bonnes valeurs de composants. Le montage est ensuite câblé et testé sur une Labdec.

3. La carte hacheur 4 quadrants

Au premier semestre nous avons étudié des cartes hacheurs 1 quadrant, nous en avons deux, une pour chaque moteur. Nous avons pu voir leurs nombreux défauts comme le fait qu'elles n'étaient pas réversibles. Nous étions censés faire une nouvelle carte hacheur au semestre 1, qui remplacerait les deux cartes dégradées, tout en réglant leurs défauts. Faute de temps nous n'avons pas pu faire cette nouvelle carte, c'est pour cela que nous nous y sommes mis au second semestre.

Nous avons donc dû trouver du temps pour concevoir cette carte qui nous suivait depuis le premier semestre. Comme d'habitude pour la création d'une carte, nous avons d'abord dû créer son schematic (à retrouver en grand en annexe), Quentin FRANCES se chargea de cette partie hors des séances de SAE. Il s'agit de la partie la plus importante car si le schematic est faux la carte ne fonctionnera pas, et sera bonne pour la poubelle. Tout d'abord, pour réaliser cette partie, il faut comprendre ce qu'est une carte hacheur 4 quadrants. Il s'agit d'une carte pouvant contrôler les deux moteurs dans les deux sens de rotation possible. Pour cela, nous utilisons un système simple et très souvent utilisé : Il s'agit d'un pont de Graetz, un par moteur, c'est-à-dire un assemblage spécifique de diodes qui permettent de bloquer ou bien de laisser passer le courant selon la situation.

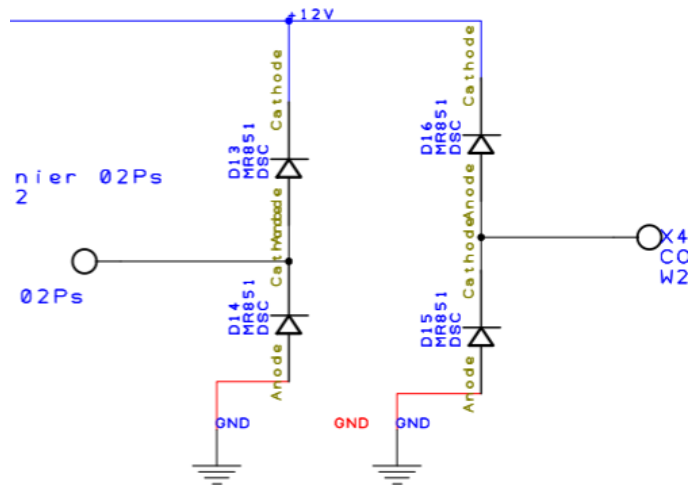


Figure 1 : Pont de Graetz - Hacheur

Ce système sert à déterminer dans quel sens le moteur tournera, mais ce n'est pas tout, nous avons aussi besoin d'un composant qui permettra la communication entre nos deux ponts de Graetz et les moteurs en passant par la carte commande. Il s'agit du L298, un composant à la composition très complexe, mais il faut retenir qu'il permet que tout fonctionne car il relie le système à la carte commande donc aux moteurs.

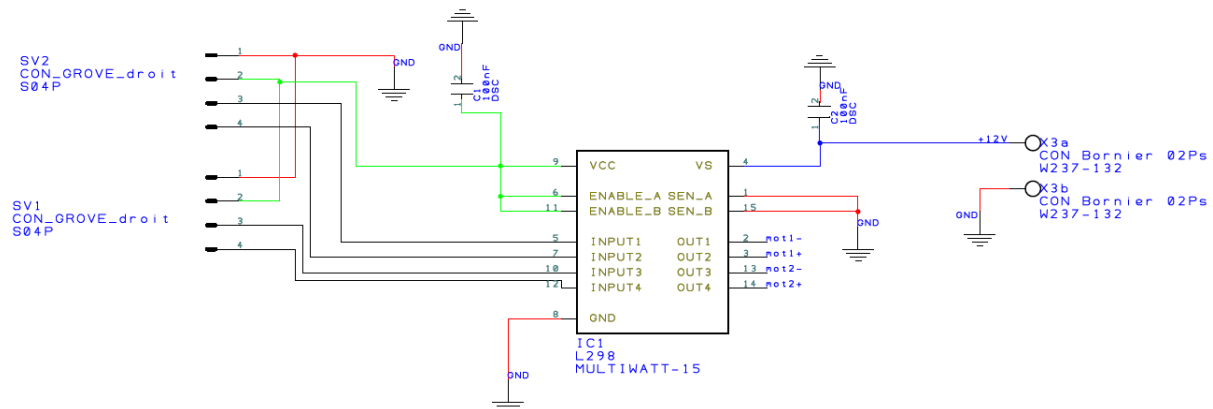


Figure 2 : L298 - Hacheur

La partie la plus complexe était donc finie, nous pouvions donc passer à la partie PCB, pour cette partie c'est Corentin CIAMPI qui s'en est chargé.

La carte hacheur 4 quadrants n'est pas des plus faciles pour la conception de son PCB, car elle est composée de beaucoup de composants qui vont presque tous au même endroit. Par conséquent beaucoup de fils d'ange se superposaient, ce qui n'aidait pas du tout à sa conception, pour régler cela le plus important était le placement au préalable de tous les composants pour que le moins possible de fils d'ange se superposent. Il ne restait alors plus qu'à tout relier.

Voici donc ci-dessous le PCB fini de la carte hacheur 4 quadrants :

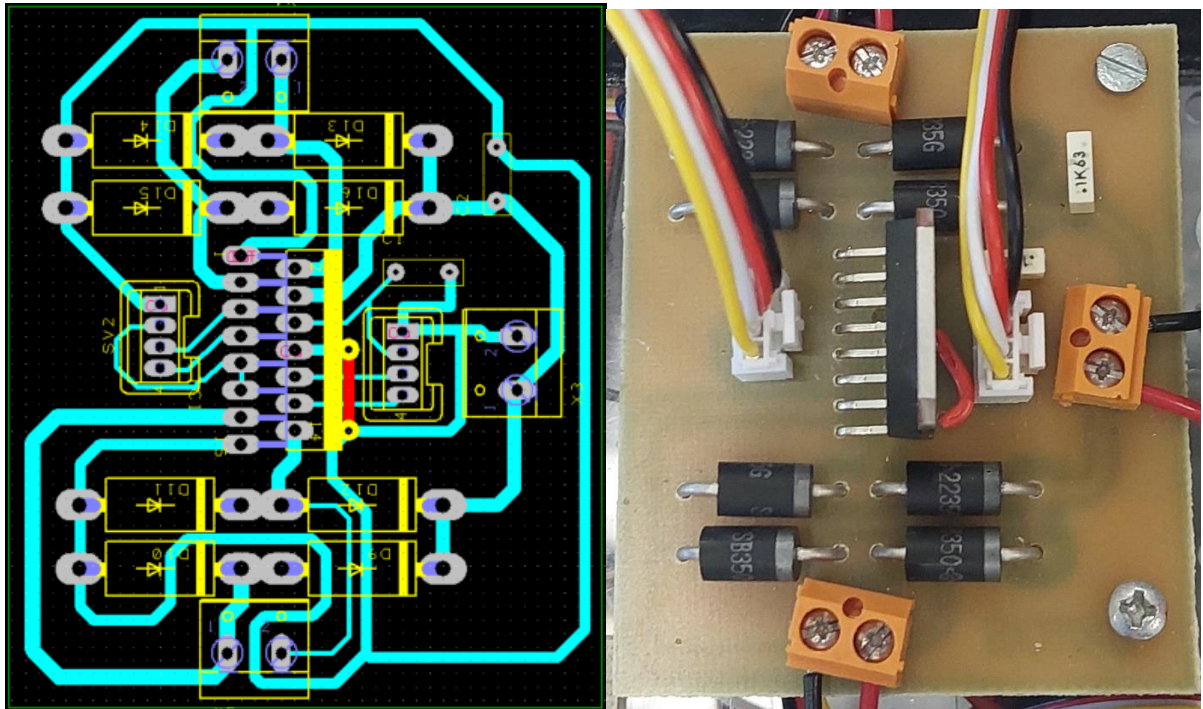


Figure 3 : PCB - Hacheur

II) La carte mezzanine

1. Comment créer les tensions nécessaires ?

Le plus gros travail que nous avons dû réaliser durant ce semestre a été de réaliser la carte mezzanine. Comme expliqué plus tôt, nous avons eu un nouveau cahier des charges avec de nouvelles épreuves, et pour cela, nous devons réaliser une nouvelle carte avec un capteur infrarouge, 2 capteurs ultra-sons, et un écran LCD. Pour le filtrage du signal infrarouge (nous rentrerons dans les détails plus tard) il nous faut utiliser des amplificateurs opérationnels (AOP). Et qui dit AOP dit alimentation de ces derniers, en +5V/-5V pour nous. Il nous a donc fallu apprendre à « fabriquer » du -5V. Nous aurons également besoin de 3,3V.

a. Créer du -5V.

Nous allons donc avoir besoin de -5V. Le +5V nous est fourni par la carte commande. Afin de créer cette tension, nous allons utiliser un TCL7660. C'est un composant qui, en mettant une tension entre 1,5V et 10V, renvoie cette même tension en négatif. On utilise des condensateurs de découplage afin de stabiliser la tension.

Ci-dessous le schéma du montage :

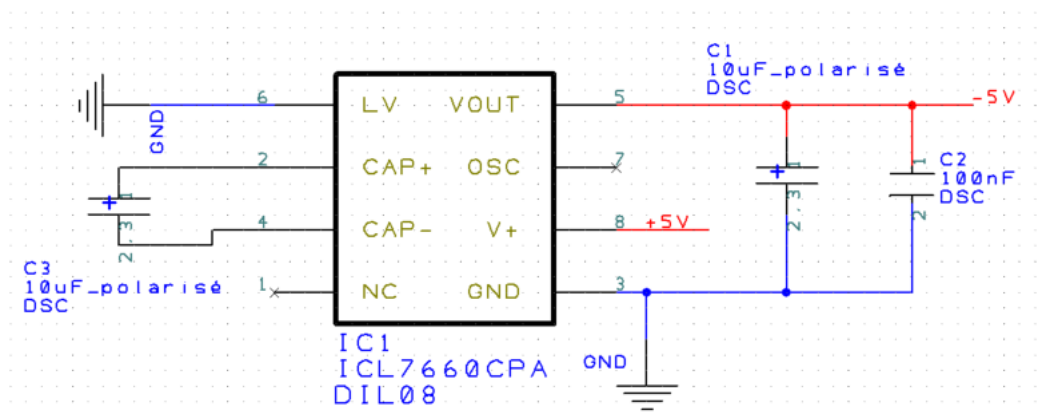


Figure 4 : TCL7660 -5V

b. Créer du 3V3

Il nous faut également posséder du 3,3V. Pour cela nous utilisons un L78L33, qui permet, en lui mettant du 5V en entrée, de sortir du 3V3.

Pour les mêmes raisons que précédemment, nous ajoutons 2 condensateurs de 100nF chacun.

Ci-dessous, le schéma du montage :

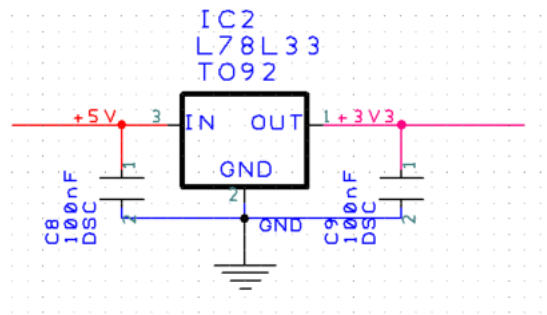


Figure 5 : L78L33 - 3V3

2. Le traitement du signal

La partie la plus importante à réaliser sur la carte mezzanine est le traitement du signal infrarouge. Le but de l'épreuve ici est de détecter une balise et de s'en rapprocher. Cependant, 3 balises sont disposées devant le robot : une de 500 Hz, une de 5kHz, et une de 500kHz. Le but est de détecter la balise émettant du 5kHz. Pour cela, il faut donc filtrer le signal reçu par le capteur infrarouge.

Voici le schéma bloc du montage que nous devons réaliser :

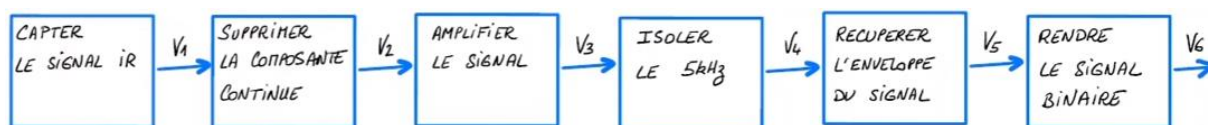


Figure 6 : Schéma bloc IR

Nous allons donc étudier chaque partie de ce schéma, et donc avancer au fur et à mesure dans le schematic de la carte mezzanine (disponible entièrement en annexe).

Avant cela, il est nécessaire de faire un petit aparté. Tout au long de ce filtrage, nous allons utiliser bon nombre d'AOP, plus exactement 6. Nous avons choisi d'utiliser des TL084, des circuits intégrés qui contiennent chacun 4 AOP. Ceci nous a permis de limiter le nombre de composants, 2 au lieu de 6.

a. Capter le signal infra-rouge

Afin de capter le signal infrarouge, nous utiliserons le capteur SFH309FA. C'est un phototransistor qui se comporte comme une source de courant qui délivre plus ou moins de courant en fonction de l'éclairement infrarouge qu'il reçoit. Afin de récupérer une tension, nous mettons ensuite un montage AOP inverseur et obtenons ce signal en sortie de ce dernier (en bleu) :

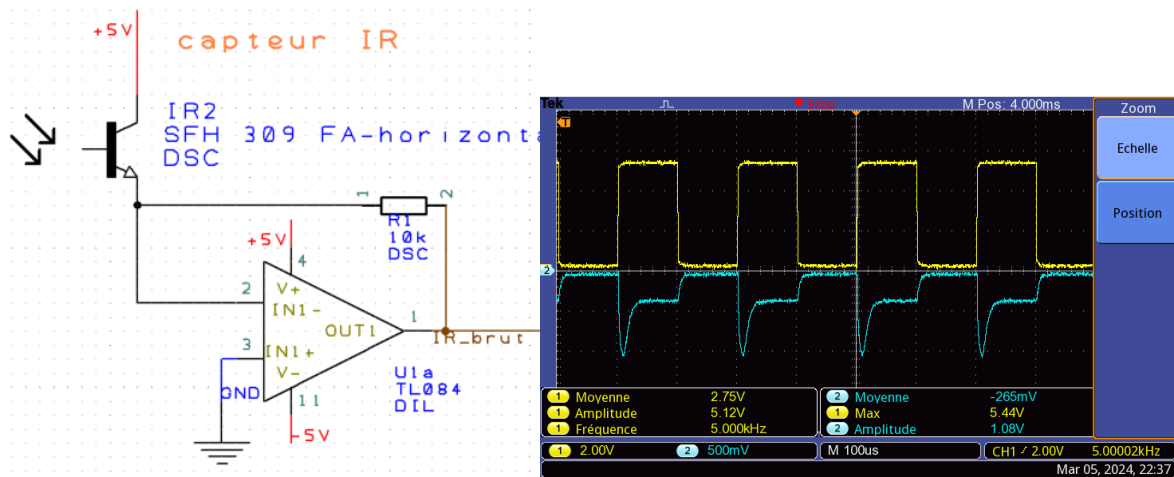


Figure 7 : Capter le signal - IR

Le signal est donc actuellement inversé, mais il le sera de nouveau plus tard dans le filtrage grâce à un deuxième montage inverseur et sera donc redressé.

b. Supprimer la constante continue

Le premier élément du filtre est la suppression de la constante continue, qui nous est ici inutile. Pour cela, nous devons réaliser un filtre passe-haut, qui laissera donc passer les fréquences supérieures à la fréquence de coupure, et qui filtrera, avec une pente de -20dB par décade, les fréquences inférieures.

Nous choisissons une fréquence de coupure $f_c = 100$ Hz, puis fixons le condensateur à 100nF et calculons R.

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi * 100 * 100 * 10^{-9}} = 15,9k\Omega$$

Nous prenons donc $R = 16k\Omega$, et nous aurons une fréquence de coupure $f_c = 99,47$ Hz.

Ce qui nous donne ceci :

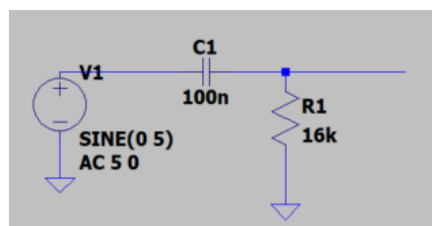


Figure 8 : Filtre passe-haut - IR

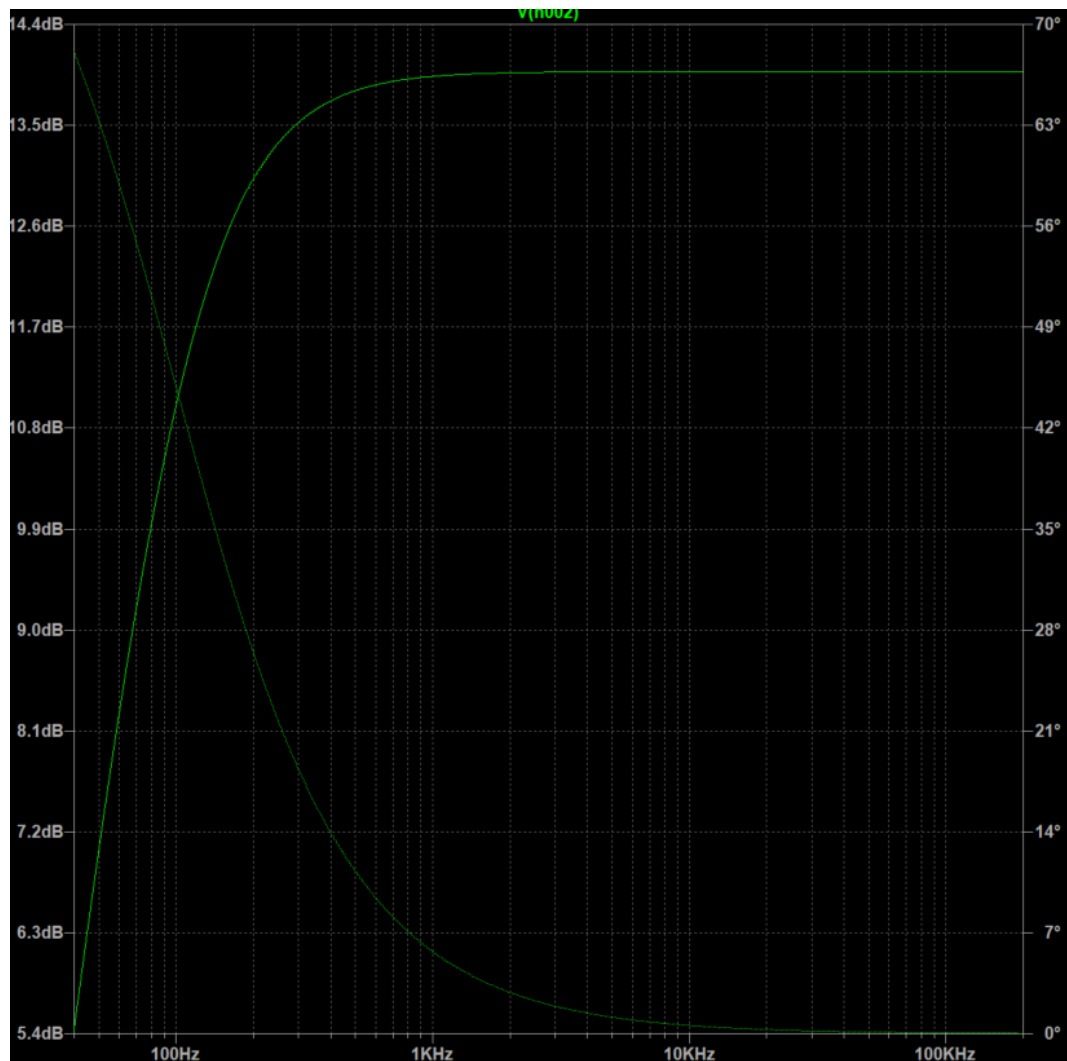


Figure 9 : Résultat filtre passe-bas - IR

c. Amplifier le signal

Pour réussir à capter le signal en étant suffisamment éloigné, il faut amplifier le signal. Pour cela, on utilise un AOP non-inverseur avec un potentiomètre afin de pouvoir régler l'amplification au besoin. Ci-dessous, la relation afin de calculer un gain pour un AOP non-inverseur.

$$\frac{V_s}{V_e} = 1 + \frac{R_2}{R_1}$$

Nous décidons de faire un gain de 2, on veut donc $\frac{V_s}{V_e} = 2$. Pour cela on fixe le potentiomètre (R_2) à 10kΩ. On obtient alors ceci :

$$R_2 = (2 - 1)R_1 = R_1$$

Dans le montage ci-dessous R1 correspond à R3 et R2 correspond à R4.

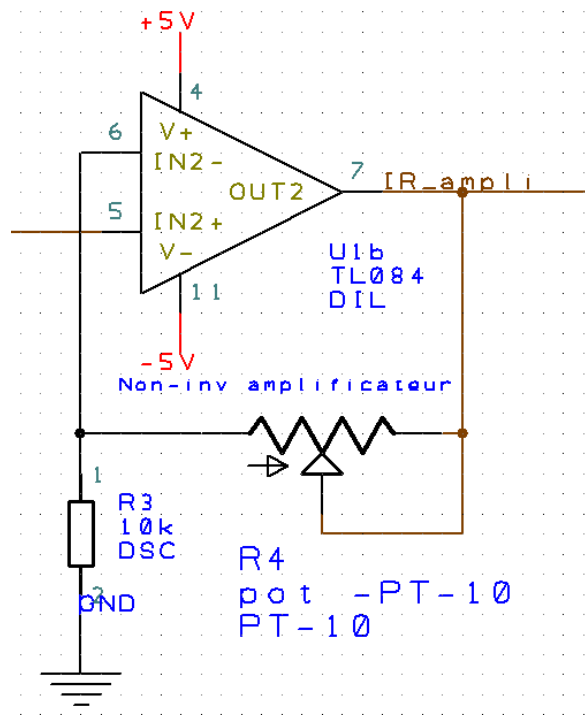


Figure 10 : Amplification du signal - IR

d. Isoler le 5kHz

On rentre à présent dans la partie la plus importante de ce schéma bloc. Il nous faut ici filtrer le 5kHz, c'est-à-dire réaliser un filtre passe-bande, de fréquence $f_0 = 5\text{kHz}$ avec un gain idéalement autour des 20dB. Un filtre passe-bande se compose d'un passe-haut en cascade avec un passe-bas, avec un AOP suiveur entre les 2. Etant donné que la fréquence de coupure correspond au gain maximum moins 3dB, il nous faut choisir des fréquences relativement éloignées de f_0 . Nous choisissons donc pour le passe-haut une fréquence de 3kHz et pour le passe-bas, une fréquence de 7kHz.

On fixe dans les 2 cas les condensateurs à 10nF, afin de calculer R.

Pour le passe-haut :

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi * 3000 * 10 * 10^{-9}} = 5305\Omega$$

On choisira donc une résistance de 5k1Ω, et on aura une fréquence de coupure de 3120Hz.

Pour le passe-bas :

$$R = \frac{1}{2\pi * 7000 * 10 * 10^{-9}} = 2274\Omega$$

On choisira donc une résistance de $2k2\Omega$, et on aura une fréquence de coupure de 7234 Hz.

Après étude (tant en simulation que sur Labdec), nous nous rendons compte qu'un filtre d'ordre 1 ne suffit pas, car l'atténuation à 500Hz et à 50kHz n'est pas suffisante. Il faut donc faire un filtre d'ordre 2, ce qui consiste à mettre 2 filtres passe-haut et 2 filtres passe-bas en cascade.

Voici donc le montage utilisé et son résultat :

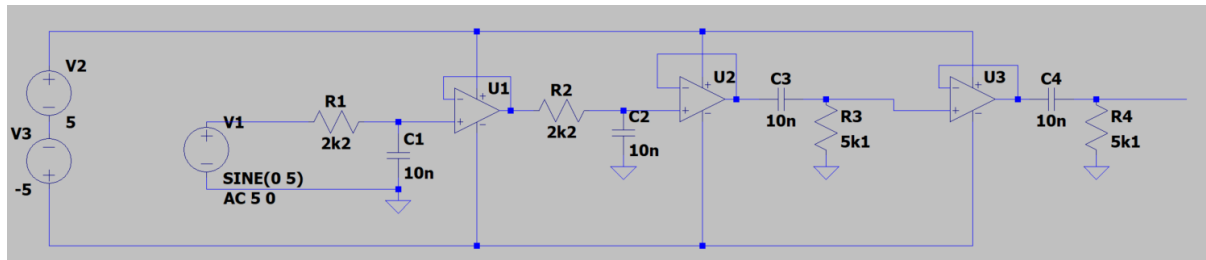


Figure 11 : Filtre passe-bande - IR

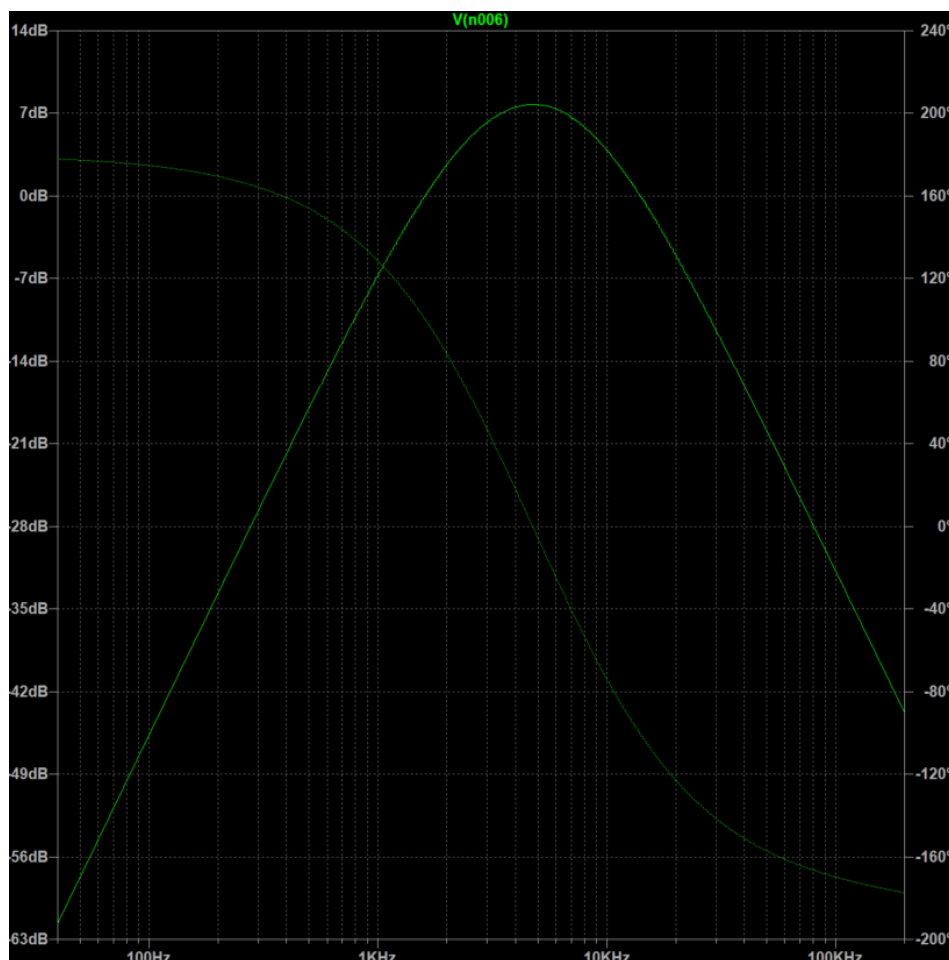


Figure 12 : Résultat filtre passe-bande - IR

Grâce à ce filtre d'ordre 2, nous réussissons donc à filtrer correctement le signal afin de récupérer le signal de la balise de 5kHz.

e. Récupérer l'enveloppe du signal

L'avant dernière étape est de récupérer l'enveloppe du signal afin d'exploiter une tension proche d'une tension continue, et donc diminuer les fluctuations de la tension. Pour cela on utilise un détecteur de crête que voici :

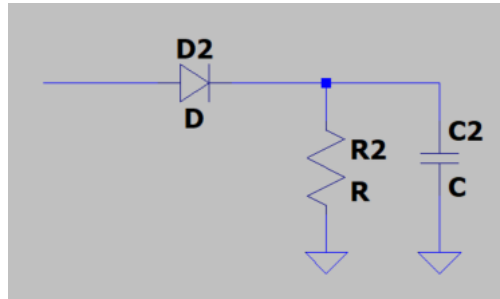


Figure 13 : Détecteur crête imparfait - IR

On injecte un signal sinusoïdal à l'entrée du montage. Durant l'alternance positive, la diode est passante et charge le condensateur. Durant l'alternance négative, la diode devient bloquante et le condensateur se décharge dans la résistance. Cependant, ce détecteur de crête n'est pas parfait car la tension seuil de la diode n'étant pas 0V, nous n'obtenons pas l'amplitude entière du signal comme on peut le voir si dessous (en rouge):

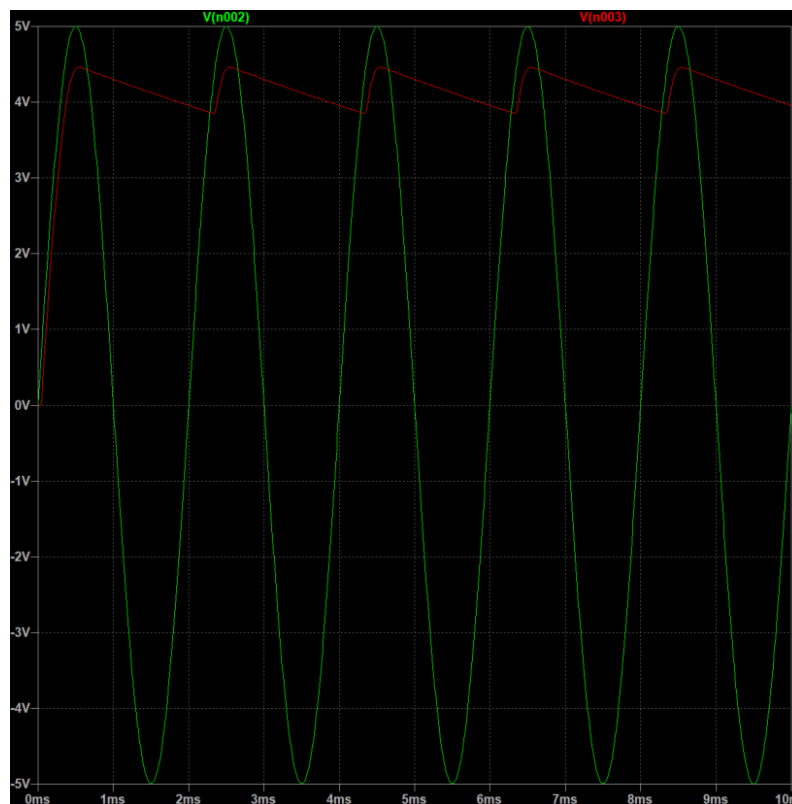


Figure 14 : Résultat détecteur crête imparfait - IR

Pour éviter cela, on rajoute un AOP suiveur, qui va faire en sorte que la tension seuil de la diode soit nulle. On aura alors un signal qui épouse complètement le signal d'entrée.

Afin de calculer les bonnes valeurs de composants, il faut que la constante de temps τ soit 10 fois supérieure à la période $T = 1/f$, avec $f = 5\text{kHz}$. On veut donc $\tau > 2\text{ms}$. Pour cela on fixe $R = 30\text{k}\Omega$ et $C = 220\text{nF}$. On obtient :

$$\tau = RC = 30000 * 220 * 10^{-9} = 6\text{ms}$$

Cette valeur nous permet d'être largement supérieur à 2ms sans pour autant être trop élevée, ce qui pourrait poser problème car le temps de décharge du condensateur serait alors trop long.

On obtient le montage suivant :

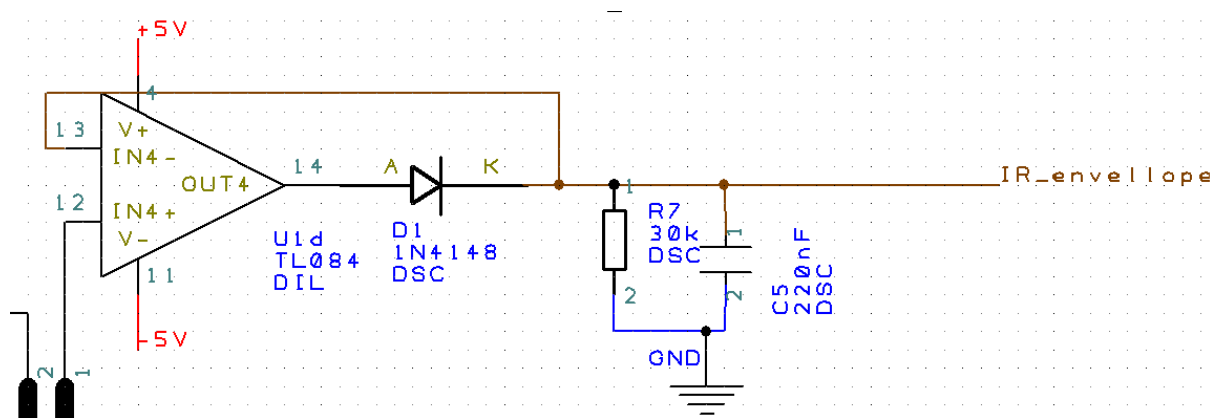


Figure 15 : Détecteur crête avec diode sans seuil – IR

f. Rendre le signal binaire

Pour des soucis de simplicité, nous allons rendre le signal binaire afin qu'il soit lisible sur une entrée digital du microcontrôleur.

Pour réaliser cela, nous allons utiliser le composant LM311. C'est un comparateur qui va donc comparer sa tension d'entrée avec une tension pré-réglée. Si la tension d'entrée est supérieure à la tension pré-réglée, le composant va laisser passer un signal, et si il est inférieur il ne laissera rien passer. On rajoute ensuite une résistance de Pull-up, reliée à du 3V3, afin de pouvoir l'envoyer sur le microcontrôleur.

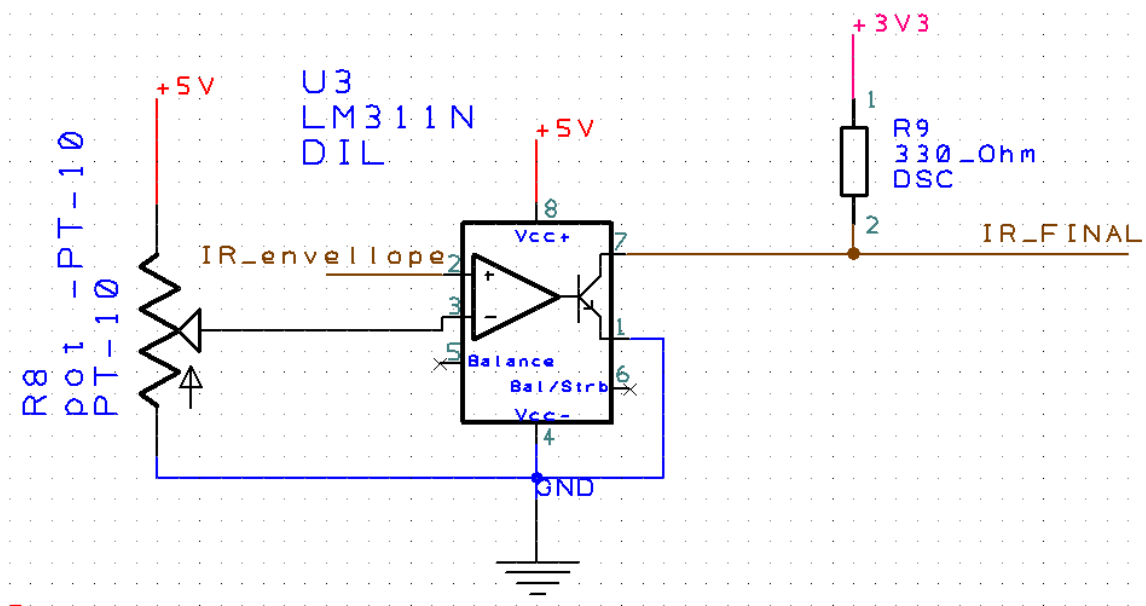


Figure 16 : LM311 – IR

3. Les capteurs ultra-sons

Une des épreuves de ce semestre 2 est de sortir d'un labyrinthe qui n'est évidemment pas connu à l'avance. Pour cela nous allons utiliser des capteurs ultra-sons, les télémètre SRF04. Ce sont des composants alimentés en +5V.



Figure 17 : SRF04

Le principe de fonctionnement est simple. On envoie une impulsion sur la broche Trigger, ce qui va envoyer une onde ultra-son dans l'air. On déclenche en même temps un timer. Lorsque l'on détecte un front montant sur Echo, cela veut dire que l'onde est revenue au télémètre. On arrête donc le timer.

Il suffit ensuite de mesurer ce temps et, comme on connaît la vitesse du son, on peut en déterminer la distance aller-retour entre le télémètre et l'obstacle. Comme on veut détecter un front montant sur la broche Echo, il faut que celle-ci soit câblée sur une broche interruptible. Voici le programme afin de calculer une distance avec le SRF04 :

```
1  #include "mbed.h"
2
3  DigitalOut trig(D4);
4  InterruptIn echo(D5);
5  Timer timer;
6  Ticker tic;
7  float mesure;
8  void echo_rise();
9  void echo_fall();
10 void tck();
11
12 int main(){
13     echo.rise(&echo_rise);
14     echo.fall(&echo_fall);
15     tic.attach(&tck, 1);
16     while(true){
17         printf("val = %lg\n\r", mesure);
18         printf("distance : %lg \n\r\n", (340*mesure)/2);
19         wait(0.1);
20     }
21 }
22
23 void tck(){
24     trig.write(1);
25     wait_us(20);
26     trig.write(0);
27 }
28
29 void echo_rise(){
30     timer.start();
31 }
32
33 void echo_fall(){
34     timer.stop();
35     mesure = timer.read();
36     timer.reset();
37 }
```

Figure 18 : Code 1 - SRF04

Une autre méthode nous est disponible et va permettre de simplifier le code. Il existe une bibliothèque du nom de SRF05.h qui réalise tout le code précédent automatiquement. C'est ce que nous allons utiliser pour la suite du code, pour une question de simplicité. Voici donc ce que cela donne :

```
#include "mbed.h"
#include "SRF05.h"

SRF05 Capt_avant(A5, D4);
SRF05 Capt_gauche(PT30, D5);

int main(){
    while(true){
        printf("Capteur avant : %lg", Capt_avant.read());
        printf("Capteur gauche : %lg\n\r", Capt_gauche.read());
    }
}
```

Figure 19 : Code avec bibliothèque - SRF04

Ces capteurs ne sont pas directement soudés sur la carte mezzanine. Ce que l'on soude sur cette dernière sont des connecteurs tulipes qui permettront ensuite de connecter les 2 capteurs. Le schematic est donc très simple, le voici :

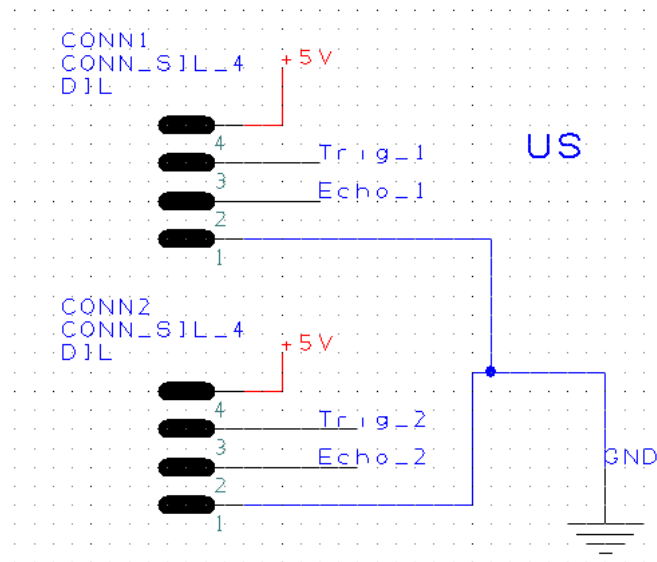


Figure 20 : Schematic - SRF04

4. L'écran LCD

Au semestre 1, nous avons conçu une carte IHM afin de pouvoir interagir avec le robot. Au semestre 2, nous supprimons cette carte pour la remplacer par la carte mezzanine. Cependant, dans le but de conserver un affichage d'information nous intégrons un écran LCD. Il permettra d'afficher des informations telles que le mode dans lequel est le robot, les valeurs des différents capteurs etc...



Figure 21 : Grove-LCD RGB Backlight

Le Grove-LCD RGB Backlight est un écran qui communique en I2C avec le microcontrôleur, c'est-à-dire avec une broche SCL (une clock) et une broche SDA. C'est via cette dernière broche que nous envoyons des informations à l'écran. On retrouvera sur la broche 1 du Grove le GND, sur la broche 2 le 5V, sur la broche 3 le SDA et sur la broche 4 le SCL. Pour la broche 3 et 4 on rajoute une résistance de Pull-up de 4k7Ω chacune.

Ci-dessous le schéma de câblage de l'écran LCD :

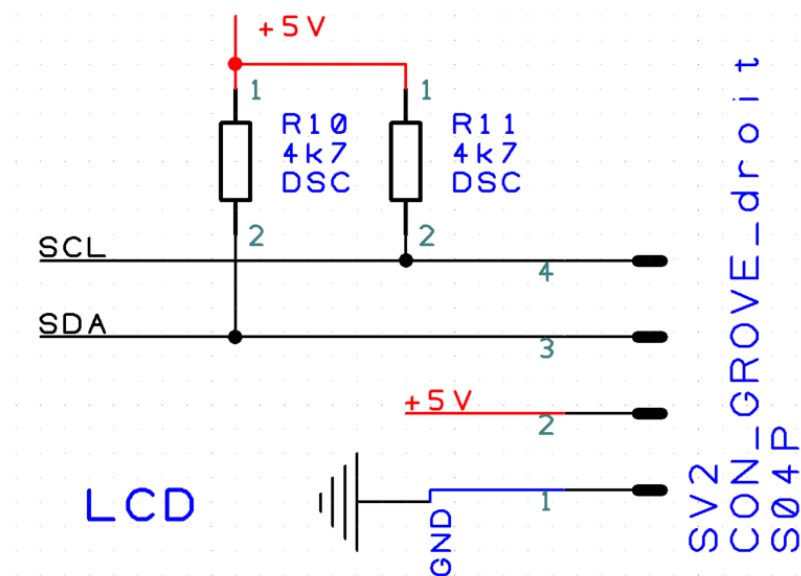


Figure 22 : Schematic LCD

Voici un code tout simple afin d'afficher « Hello world ! » sur l'écran LCD :

```

1  #include "Grove_LCD_RGB_Backlight.h"
2  #include "mbed.h"
3
4  Grove_LCD_RGB_Backlight lcd(D14,D15);
5
6  int main(){
7      char tab[20];
8      while(true){
9          lcd.clear();
10         lcd.locate(0,0);
11         lcd.setRGB(255,255,255);
12         sprintf(tab, "Hello world !");
13         lcd.print(tab);
14     }
15 }
```

Figure 23 : Code LCD - Hello World !

char tab[20] ; créer un tableau qui sera utilisé pour afficher le message, il sera rempli par la suite.

lcd.clear() ; Efface l'écran. Cela permet de remettre l'écran à 0 au cas où il y aurait d'autre message affichés

lcd.locate(0,0) ; veut dire que l'on positionne le message en colonne 0, ligne 0.

lcd.setRGB(255,255,255) ; signifie que l'on fixe la couleur du fond, ici en blanc.

sprintf(tab, « Hello world ! ») ; Injecte le message « hello world » dans le tableau créé précédemment.

lcd.print(tab) ; On affiche le message sur l'écran.

5. Liste des composants et répartition du travail

Voici la liste complète des composants présents sur la carte Mezzanine.

COMPOSANT	INFORMATION
CONDENSATEUR 10μF (x2)	Condensateur de découplage
CONDENSATEUR 100nF (x4)	Condensateur pour le filtrage
CONDENSATEUR 220nF	Condensateur pour detecteur crête
CONDENSATEUR 10nF (x4)	Condensateur pour le filtrage
LM311	Comparateur
TL084 (X2)	Amplificateurs Opérationnels
GROVE DROIT	Grove pour le LCD
HE10	Connection avec la carte commande
TCL7660	Convertisseur +5V/-5V
L78L33	Régulateur de tension 5V/3v3
SFH308FA	Photo-transistor
RESISTANCE 10kΩ (x2)	
RESISTANCE 16kΩ	
RESISTANCE 30kΩ	
POT-PT-10 (x2)	Potentiomètre 10k Ω
RESISTANCE 330 Ω	Résistance de Pull-Up
RESISTANCE 4k7Ω (x2)	
RESISTANCE 2k2Ω (x2)	
RESISTANCE 5k1Ω (x2)	
CONN SIL4 (x2)	Connecteur tulipe pour les ultra-sons
CONN SIL2 (x2)	Connecteur tulipe pour le passe-bande
DIODE 1N4148	Diode pour detecteur crête

L'étude du filtre a été réalisée par FRANCES Quentin et CASA Cost, celle de l'écran LCD par CIAMPI Corentin et BARROIS Victor, et celle des capteurs ultra-sons par CASA Cost et BARROIS Victor.

Le schematic et les soudures ont été réalisées par FRANCES Quentin et le PCB par CIAMPI Corentin, que vous pouvez retrouver ci-dessous (et en grand en annexes).

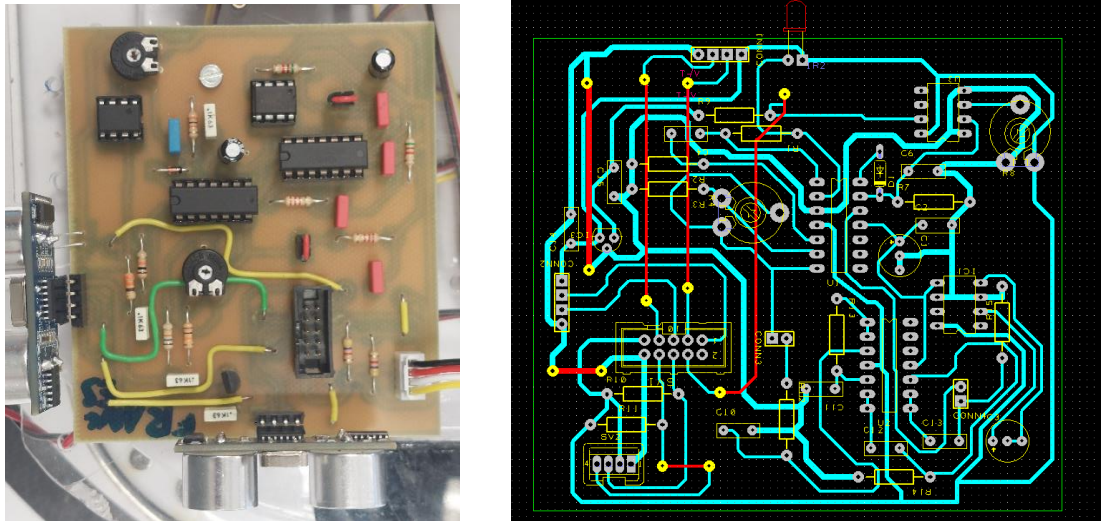


Figure 24 : Carte Mezzanine – Soudure et PCB

III) Les compléments

1. Le module Grove MP3

En plus du module Bluetooth et pour ajouter un peu de challenge, un module Grove MP3 est ajouté sur le robot. Il permet de lire des sons ou des bruitages stockés dans une carte SD. Le module peut être utilisé en même temps que le robot avance et fait les épreuves, ce qui laisse le choix sur la créativité et sur la sélection des sons émis (un son lorsque le robot avance et un son lorsque le robot s'arrête par exemple).



Figure 25 : Grove MP3

Voici ses caractéristiques :

- Décodeur audio WT2003S-20SS
- Prend en charge les fichiers audio au format MP3
- Taux d'échantillonnage : 8 ~ 48KHz / débit binaire : 8 ~ 320Kbps
- Supporte jusqu'à 32 Go sur la carte SD
- Supporte le haut-parleur et les écouteurs audio de sortie en même temps
- Compatible avec la plate-forme 3.3V et 5V
- Prise en charge du réglage du volume à 32 niveaux

Le protocole de communication utilisé entre le microcontrôleur MBED et le module Grove MP3 est l'UART: il faut donc croiser les Rx et les Tx. De plus, nous utilisons les broches PTE22/PTE23 de la carte MBED pour pouvoir brancher le module MP3 (sur notre carte commande ces pins sont disponibles sur le CONN3).

On se retrouve alors avec cette configuration :

Fil noir : GND

Fil rouge : 5V

Fil Blanc : Rx

Fil Jaune : Tx

Afin de connecter le Grove MP3 à la carte commande, nous devons créer une mini carte adaptatrice afin d'inverser des fils, sans quoi la connexion serait impossible.

Voici le schematic et le PCB de cette petite carte (créée par Quentin FRANCES) :

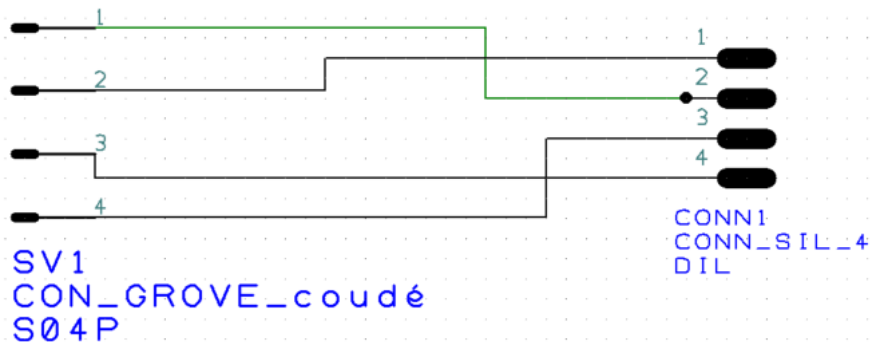


Figure 26 : Schematic mini carte adaptatrice

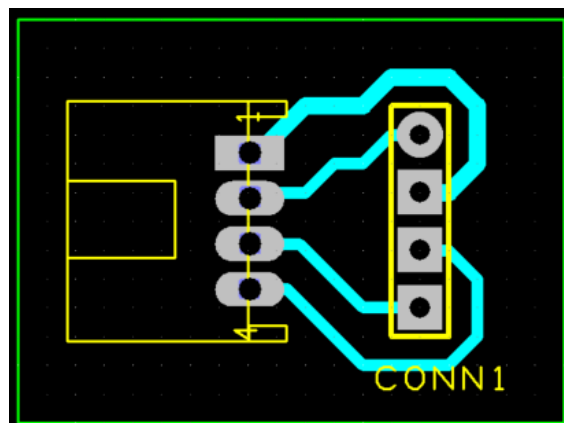
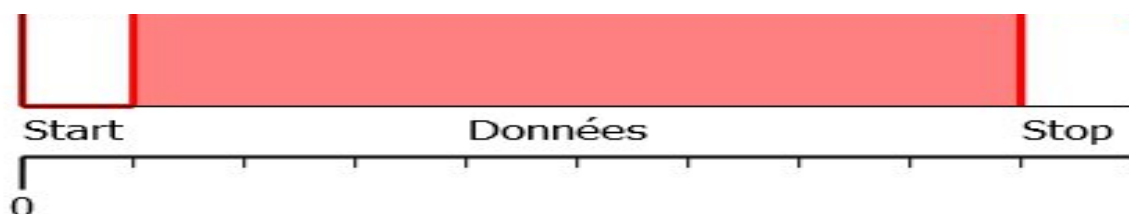
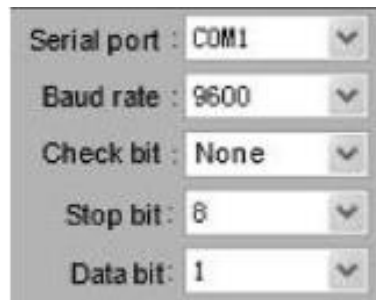


Figure 27 : PCB mini carte adaptatrice

Le module MP3 fonctionne avec un système de bits. C'est-à-dire qu'on va avoir 1 bit de démarrage, 8 bits de données et 1 bit d'arrêt.



De plus, comme le module MP3 utilise une communication UART / série nous devons absolument définir correctement les paramètres du port série sur Teraterm, comme indiqué sur l'illustration ci-après.



Serial port : COM1
 Baud rate : 9600
 Check bit : None
 Stop bit : 8
 Data bit : 1

Enfin, pour utiliser le module MP3 il faut établir un tableau qui contient tous les éléments à son bon fonctionnement. L'écriture dans le tableau se fait en hexadécimal. De plus, il est important de préciser que le nombre de cases du tableau peut changer selon la commande associée au module MP3. Voici un exemple de tableau "type" que l'on peut retrouver :

Start code	Length	Command code	Parameters	Cumulative sum check	End code
0X7E	See below	See below	See below	See below	0XEF

On commence par le "Start code" qui correspond au lancement de la séquence. Il est toujours associé au code 0X7E en hexadécimal.

Ensuite nous avons le "Length" ou longueur en français qui correspond à la taille du tableau. C'est-à-dire que nous allons compter le nombre de cases après celui-ci pour définir sa taille. Dans le tableau ci-dessus on voit 4 cases ce qui correspond au code 0X04.

Le "Command code" est associé à la commande que nous voulons lui donner. Il en existe près de 28 allant de la commande A2 à CB (vous pourrez retrouver un tableau avec l'ensemble des commandes en annexe).

Le "Parameters" n'est pas présent dans tous les tableaux. En effet, il correspond à une variable qui est utilisée notamment pour augmenter ou baisser le volume. Nous allons donc jouer sur ce paramètre et le faire varier.

Le "Cumulative sum check" est très important car il permet de vérifier que les données binaires ont bien été transmises, et qu'il n'y a pas eu de pertes.

Enfin, le "End code" qui correspond à la fin de séquence permet de clôturer la communication. Il est toujours associé au code 0XEF en hexadécimal.

A présent, voyons donc comment est-ce que l'on code tout cela.

Prenons tout d'abord l'exemple du tableau ci-dessus, ce qui nous donne ceci :

```
void fonction(int parametre){
    char tab[6] = {0x7E, 0x04, commande, parametre, 0, 0xEF};
    tab[4] = tab[1] + tab[2] + tab[3];
    for(int i = 0; i < 6; i++){
        device.putc(tab[i]);
    }
}
```

Ici le “Cumulative sum check” se fait par une somme d’éléments du tableau, c’est pour cela qu’il y a un 0 juste après “parametre” mais il est tout à fait possible de le faire autrement en mettant directement la valeur en hexadécimal à la place du 0 et ne pas mettre la somme d’éléments. Enfin, il reste à lire tout le tableau avec une boucle for.

A présent, voyons les différentes commandes utilisées pour la SAé Robot.

Mettre en pause :

```
void playpause() {
    char tab[5] = {0x7E, 0x03, 0xAA, 0, 0xEF};
    tab[3] = tab[1] + tab[2];
    for (int i = 0; i < 5; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 28 : MP3 - Pause

Arrêter la musique :

```
void stop() {
    char tab[5] = {0x7E, 0x03, 0xAB, 0, 0xEF};
    tab[3] = tab[1] + tab[2];
    for (int i = 0; i < 5; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 29 : MP3 – Stop

Passer à la piste d’après :

```
void next() {
    char tab[5] = {0x7E, 0x03, 0xAC, 0, 0xEF};
    tab[3] = tab[1] + tab[2];
    for (int i = 0; i < 5; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 30 : MP3 – Next

Revenir à la piste d'avant :

```
void previous() {
    char tab[5] = {0x7E, 0x03, 0xAD, 0, 0xEF};
    tab[3] = tab[1] + tab[2];
    for (int i = 0; i < 5; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 31 : MP3 – Previous

Contrôler le volume :

```
void volume(int parametre) {
    char tab[6] = {0x7E, 0x04, 0xAE, parametre, 0, 0xEF};
    tab[4] = tab[1] + tab[2] + tab[3];
    for (int i = 0; i < 6; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 32 : MP3 - Volume control command

Choisir un fichier/piste en particulier :

```
void fich1() {
    char tab[9] = {0x7E, 0x07, 0xA3, 'c', 'a', 'r', 's', 0, 0xEF};
    tab[7] = tab[1] + tab[2] + tab[3] + tab[4] + tab[5] + tab[6];
    for (int i = 0; i < 9; i++) {
        device.putc(tab[i]);
    }
}

void fich2() {
    char tab[8] = {0x7E, 0x06, 0xA3, 'w', 'i', 'n', 0, 0xEF};
    tab[6] = tab[1] + tab[2] + tab[3] + tab[4] + tab[5];
    for (int i = 0; i < 8; i++) {
        device.putc(tab[i]);
    }
}
```

Figure 33 : MP3 - Specify SD card file name play

Ici les fonctions fish1 et fish2 permettent d'appeler un fichier audio MP3 stocké dans la carte SD selon un nom spécifique. Les deux fichiers sont renommés en "cars" et "win". Il suffit maintenant d'associer les actions du robot avec ces commande afin d'animer musicalement les épreuves !

2. Le module Bluetooth

Parmi les épreuves facultatives, télécommander en Bluetooth le robot fait partie de l'une d'entre elles. Plus précisément, nous devons le commander avec notre téléphone. Pour cela, on dispose d'un module HC-05, qui sera directement relié sur la carte commande où il existe un connecteur femelle 4 points libre. Concernant son fonctionnement, il suffit de relier respectivement les broches TX et RX du microcontrôleur sur les broches RXD et TXD du module Bluetooth.



Figure 34 : HC-05

Le protocole de communication de ce module étant l'UART, il faudra donc définir un objet de type Serial dans le code. Ensuite, il reste à lire les caractères reçus par le module en vérifiant régulièrement si un caractère est disponible à la lecture puis le stocker dans une variable. Pour cela, nous utilisons une application Android qui se nomme "Bluetooth Serial Controller". Pour configurer tout cela, nous devons choisir le nombre de boutons puis donner un nom à ces boutons en y associant une commande qui sera interprétée dans le code. Voici le code en question pour utiliser le module Bluetooth :

```
#include "mbed.h"

Serial mon_module_bluetooth(PTE22,PTE23);

int main(){
    char Caractere_lu;
    while(true){
        if(mon_module_bluetooth.readable()){
            Caractere_lu = mon_module_bluetooth.getc();
            printf("%c\n\r", Caractere_lu);
        }
    }
}
```

Au vu du cahier des charges très chargé et du manque de séances et donc de temps, nous avons pris la décision de ne pas utiliser ce module, pour nous concentrer davantage sur le module Grove MP3. Cependant nous pourrions envisager davantage de modifications afin de créer une petite application permettant à l'utilisateur de « piloter » le robot à distance via son téléphone.

IV) Schéma de câblage

Avec l'ajout de ces nouvelles carte nous avons dû réaliser un nouveau schéma de câblage que voici :

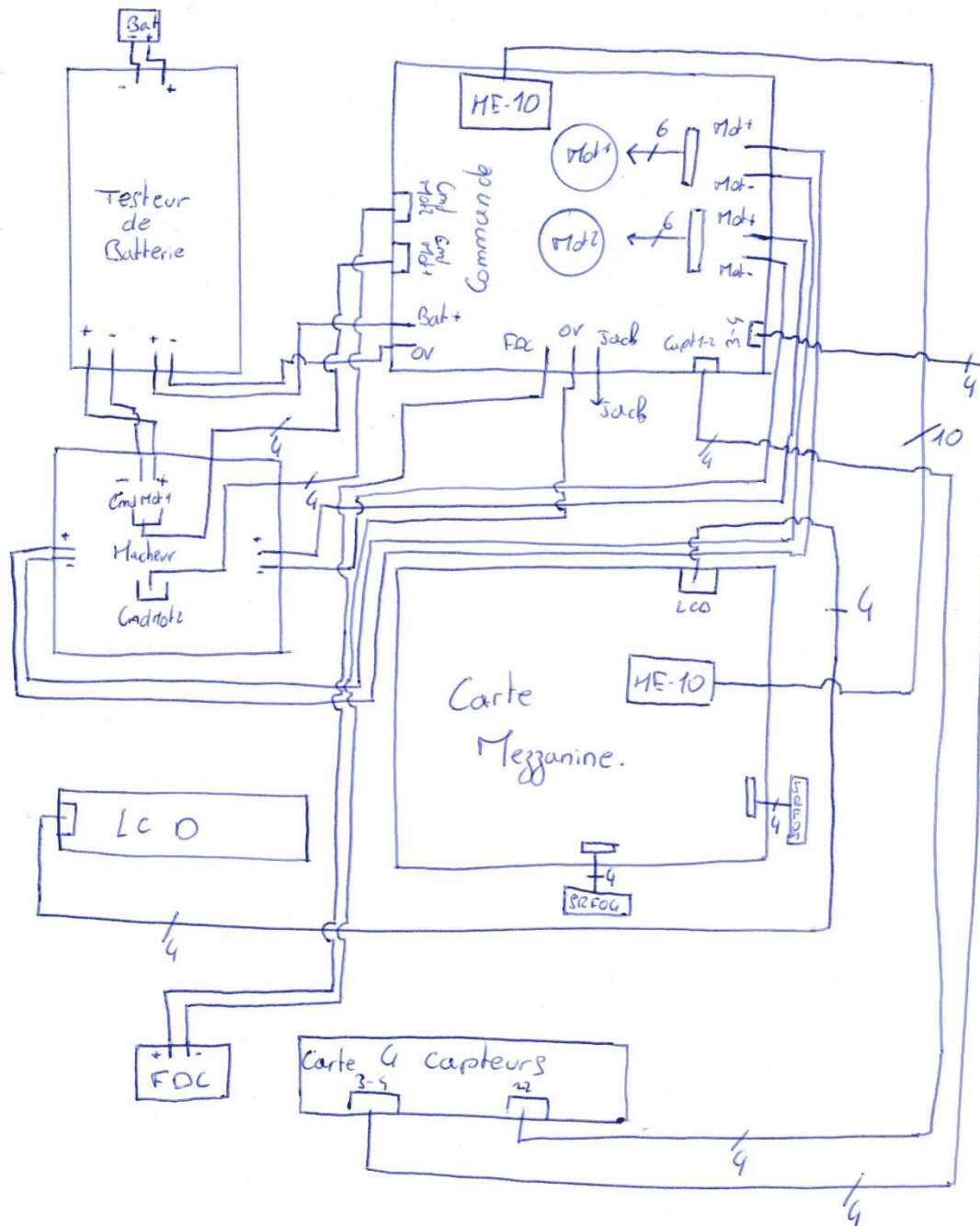


Figure 35 : Schéma de câblage

Voici donc ce que cela donne une fois câblé sur le robot :

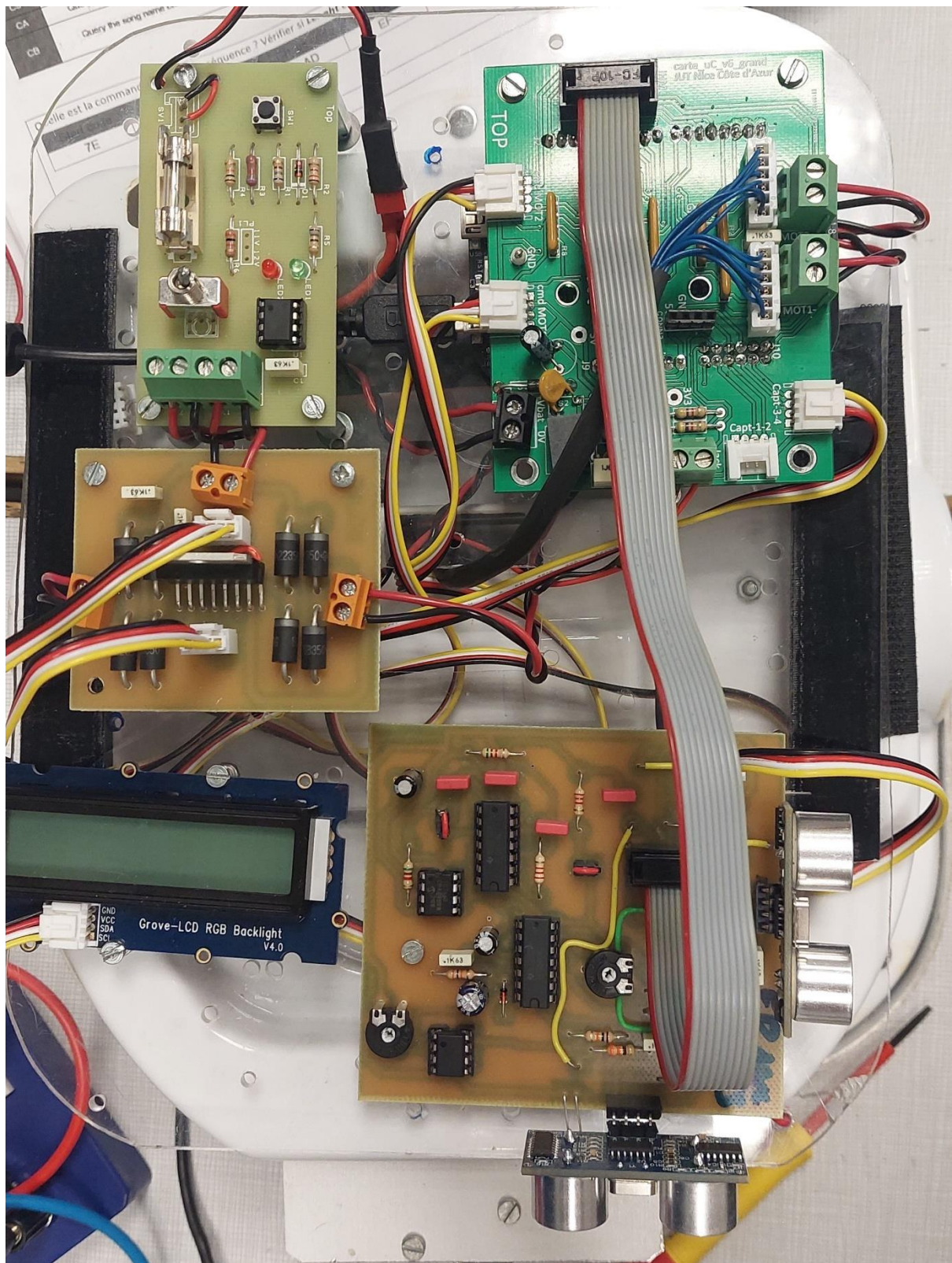


Figure 36 : Robot câblé

V) La programmation du robot

Nous allons donc à présent passer à la partie programmation du robot. C'est une partie très dense car il a fallu revoir bon nombre de programmes ainsi qu'en créer de nouveaux. Chaque épreuve étant indépendante, nous allons donc découper cette partie par thèmes/épreuves.

1. Les encodeurs

Un moteur classique n'a besoin que de 2 fils pour fonctionner : la phase et le neutre. En fonction du branchement de ces derniers le moteur tourne dans un sens ou dans l'autre. Cependant, sur les moteurs utilisés en SAE robot, nous pouvons remarquer la présence de 6 fils, soit bien plus que nécessaire. Les moteurs que l'on utilise possèdent des encodeurs, et c'est pour cela que nous avons un plus grand nombre de fils.

Un encodeur, c'est une petite carte (carte verte ci-dessous) permettant de compter les tours moteurs.



Figure 37 : Encodeurs sur un moteur

Cela fonctionne par front, c'est-à-dire qu'à chaque tour moteur, on obtient un certain nombre de fronts montants et descendants. Si l'on connaît cette caractéristique on peut donc commander les moteurs bien plus précisément. Au semestre 1 la précision était médiocre car nous n'utilisions justement pas ces encodeurs. C'est donc la première chose sur laquelle nous nous sommes penchés au début du semestre 2.

Un encodeur possède 2 fronts, un front A et un front B. Nous avons en tout 4 fronts à gérer. Le principe va être relativement simple. Si on détecte un front sur A, on va incrémenter un compteur, soit en ajoutant soit en soustrayant, et cela en fonction de la position de B. On fait donc 2 compteurs, un par moteur.

Ensuite, on compare ces 2 compteurs et s'il y a une différence alors on ralentit ou accélère le bon moteur. En faisant cela le robot est capable de tenir une ligne droite très proche de la perfection.

Ci-dessous, vous pouvez retrouver les éléments les plus importants de ce code (le code dans son intégralité sera à retrouver en annexe) :

```
void init()
{
    MOT_G.period_us(100);
    MOT_D.period_us(100);
    am1.rise(&incrementation1);
    am2.rise(&incrementation2);
}

void incrementation1(void)
{
    if(bm1.read() == 0)
        cpt1++;
    else
        cpt1--;
}

void incrementation2(void)
{
    if(bm2.read() == 1)
        cpt2++;
    else
        cpt2--;
}
```

```
void avance(int16_t dist)
{
    cpt1 = 0;
    cpt2 = 0;
    do{
        if(cpt1>cpt2)
        {
            vitesse(35.0,40.0);
        }
        else if(cpt1<=cpt2)
        {
            vitesse(35.0,30.0);
        }
        wait_us(150);
    }while(cpt1 < dist);
}
```

2. L'accéléromètre

Une des épreuves est de démarrer le robot à l'accéléromètre. Il s'agit d'une puce électronique qui va permettre de détecter les mouvements du robot sur les 3 axes, soit sur x, y et z.

Nous n'avons pas à créer cette puce car elle est directement intégrée sur le microcontrôleur FRDM-KL25Z (puce entourée en bleue ci-dessous).

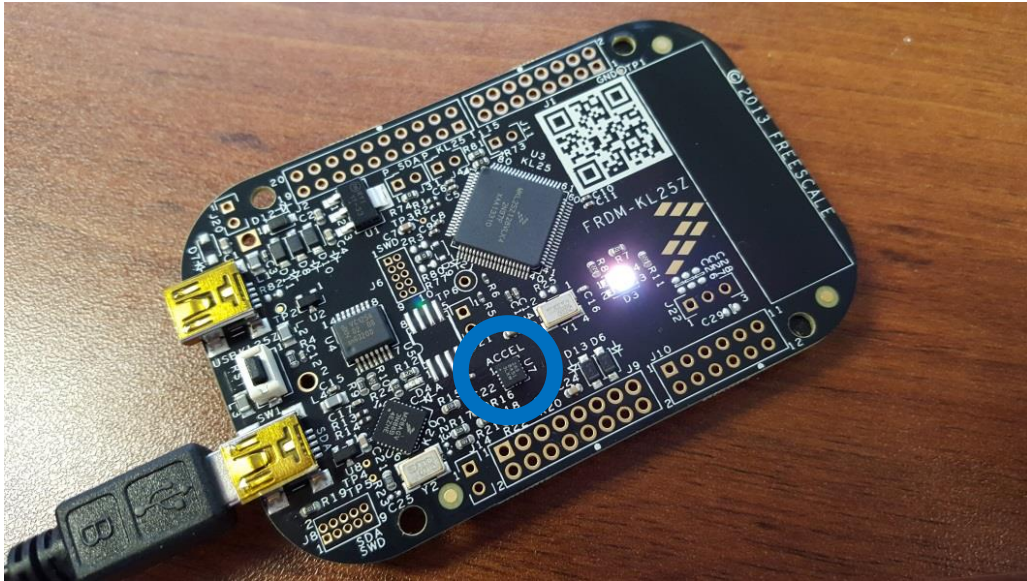
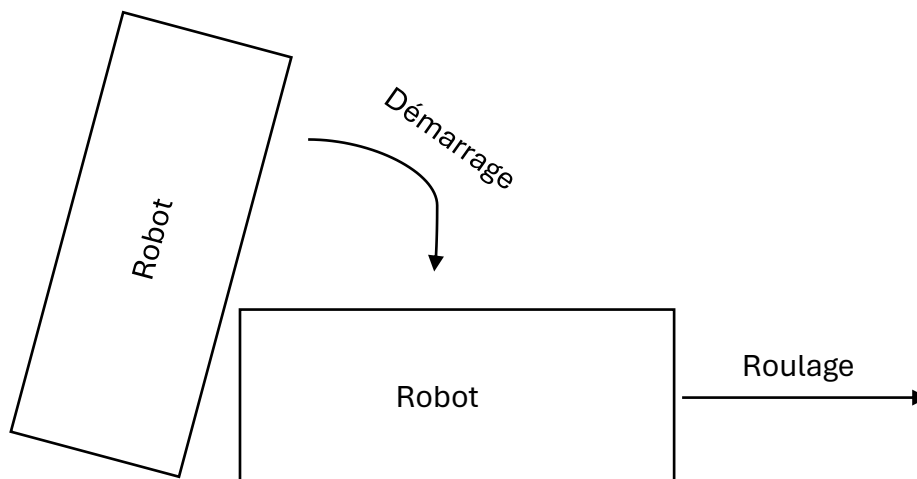


Figure 38 : Accéléromètre - FRDMKL25Z

L'utilisation de ce composant étant compliquée, il nous a été fourni un code presque complet, que nous devons modifier pour qu'il corresponde à nos envies et besoins. Nous avons fait le choix de prendre les mouvements sur Z. C'est donc en basculant le robot de la position verticale à la position horizontale qu'il se mettra à rouler, comme on peut le voir ci-dessous :



Après des test, nous observons que lorsque le robot est à l'horizontal, l'accélération sur z est supérieure à 1G. Le robot démarrera donc quand cette valeur sera supérieure à 1G.

Nous réalisons une petite machine à état, puis la codons et voici ce que cela donne :

```
switch(etat){  
    case 0 :  
        vitesse(0.0,0.0);  
        if(val_z_g > 1.0){  
            etat = 1;  
        }  
        break;  
    case 1 :  
        avance(400);  
        etat=2;  
        break;  
    case 2:  
        vitesse(0.0,0.0);  
        wait(5);  
        etat = 0;  
        break;  
}
```

L'état n°2 nous permet d'avoir 5 secondes pour récupérer le robot et le remettre en position verticale et donc l'arrêter.

(Le code entier est à retrouver en annexe)

3. Le carré

Une des épreuves est de réaliser un carré « parfait ». Cette épreuve était déjà présente au semestre 1, cependant il ne pouvait pas être parfait étant donné que nous n'utilisions pas les encodeurs.

Avec les encodeurs, il faut associer 2 fonctions, la fonction « avance » déjà vue plus tôt et la fonction « tourne » que voici :

```
void tourne(int16_t angle)  
{  
    cpt1 = 0;  
    cpt2 = 0;  
    angle = (angle*465/360)/2;  
  
    do{  
        if(cpt1 > (-1 * cpt2))  
            vitesse(35.0,-40.0);  
        else  
            vitesse(35.0,-30.0);  
        wait_us(150.0);  
    }while(cpt1 < angle);  
}
```

Il ne nous reste plus qu'à faire une séquence de ces 2 fonctions afin de réaliser un carré parfait.

4. Le suivi de ligne

Le suivi de ligne est similaire à celui du semestre 1. Le but est de calculer la différence de valeurs entre les 2 capteurs centraux puis d'agir en fonction sur les moteurs.

Si le delta est négatif, alors on ralentit le moteur droit, et à l'inverse, si le delta est positif alors on ralentit le moteur gauche.

De plus, si le capteur de fin de course est enclenché, on arrête le robot et on lui fait afficher « WIN » sur l'écran LCD.

Voici une partie du code (l'entièreté de ce dernier étant disponible en annexe) :

```
while (1) {  
    fdc = FDC.read();  
    captav = Capt_avant.read();  
    captg = Capt_gauche.read();  
    capt2 = capt_2.read();  
    capt3 = capt_3.read();  
    delta = capt2 - capt3;  
    if(delta<=0) {  
        SMOT_G.write(0);  
        SMOT_D.write(0);  
        MOT_G.pulsewidth_us(50.0);  
        MOT_D.pulsewidth_us(30.0*delta);  
    }  
    if(delta>=0) {  
        SMOT_G.write(0);  
        SMOT_D.write(0);  
        MOT_G.pulsewidth_us(30.0*delta);  
        MOT_D.pulsewidth_us(50.0);  
    }  
    else if(fdc == 0) {  
        vitesse (0.0, 0.0);  
        LCD.setRGB(255,255,255);  
        LCD.locate(6,0);  
        sprintf(tab,"WIN");  
        LCD.print(tab);  
    }  
}
```

5. Eviter un obstacle

Dans la continuité, il faut se préparer à éviter un obstacle qui se trouverait en travers du chemin de notre robot. Pour cela on utilise les télémètres SRF05.

Si on se trouve à moins de 15 centimètres d'un obstacle, on s'arrête afin de tourner de 90°.

```
case 0: if (captav <= 15 && captavav > 15) {  
    vitesse(0.0, 0.0);  
    wait(1);  
    tourne(90);  
    wait(1);  
    cpt1 = 0;  
    cpt2 = 0;  
    etat = 1;  
}
```

Une fois cela fait, on avance jusqu'à ce qu'on ne détecte plus le mur (à l'aide du deuxième capteur situé sur le flanc gauche du robot).

Lorsque c'est le cas, on exécute la séquence suivante afin de contourner l'obstacle et se remettre sur la ligne, afin de continuer le suivi de cette dernière.

```
case 1:  
    if (captg >= 40 && captgav < 40) {  
        cpt1av = cpt1;  
        avance(100);  
        wait(1);  
        tourne(300);  
        wait(1);  
        avance(200);  
        wait(1);  
        tourne(300);  
        wait(1);  
        avance(cpt1av+100);  
        wait(1);  
        tourne(90);  
        wait(1);  
        cpt1 = 0;  
        cpt2 = 0;  
        etat = 0;  
    }  
    break;
```

Durant tous ces moments, l'écran LCD affiche la distance mesurée par les télémètres.

```
printf(tab,"capteurav = %.2f",captav);  
LCD.locate(0,0);  
LCD.print(tab);  
printf(tab,"capteurg = %.2f",captg);  
LCD.locate(0,1);  
LCD.print(tab);
```

6. Détecter la balise et s'en rapprocher

Enfin, il faut détecter la balise infrarouge pour s'en rapprocher. La méthode d'approche est simple : le robot tourne sur lui-même jusqu'à capter la balise. Une fois cette dernière dans le viseur, il avance vers elle. S'il la perd, il se remet à tourner sur lui-même jusqu'à la retrouver, et ainsi de suite.

Pour faire des rotations sur lui-même, nous n'utilisons pas la fonction « tourne » mais nous mettons les 2 moteurs à la même vitesse de rotation mais en sens contraire.

Voici le code que nous utiliserons :

```
while(1)  
{  
    IR = Capteur_IR.read();  
    myled.write(1-IR);  
    if(IR ==1) {  
        vitesse(35.0,35.0);  
    }  
    else{  
        vitesse(-30.0,30.0);  
    }  
}
```

Vous remarquerez que nous ne parlons pas de l'épreuve du labyrinthe. C'est un des rares points noirs de ce semestre pour nous, car nous n'avons malheureusement pas eu le temps de le travailler.

Sachant que cette épreuve était la plus dure et n'ayant que peu de temps, nous avons priorisé tout le reste, notamment la détection de la balise infra-rouge, afin de faire le maximum d'épreuves.

VI) L'organisation dans l'équipe

Ci-dessous, un diagramme de Gantt représentant la répartition des tâches tout au long du semestre.

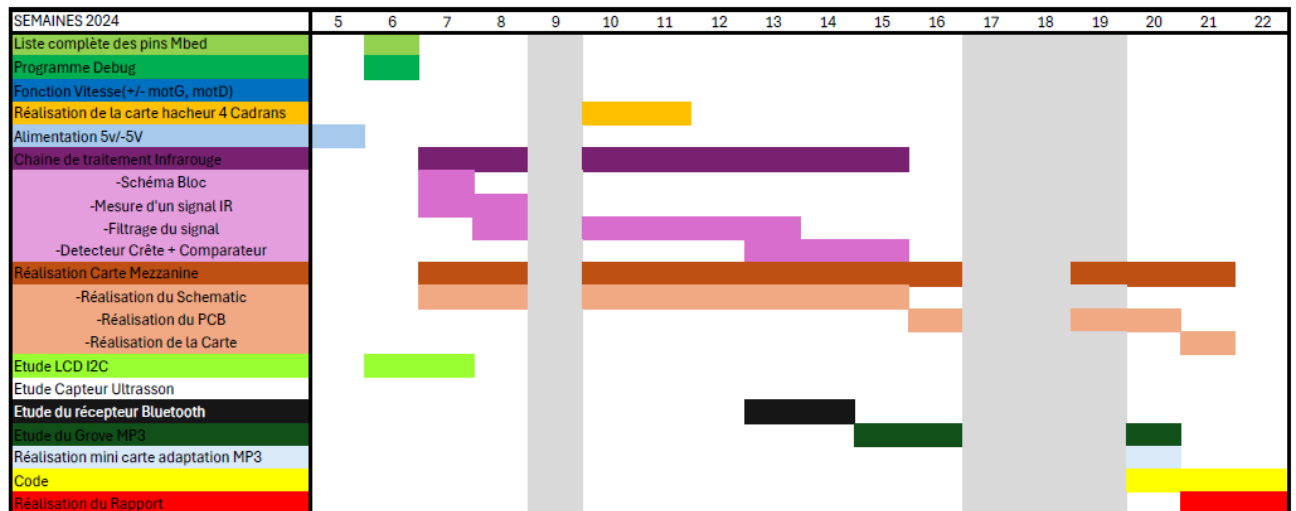


Figure 39 : Diagramme de Gantt

A préciser que nous n'avons pas de cours sur les semaines 21 et 22, et que nous sommes venus en plus sur notre temps libre, parfois durant l'entièreté de la journée.

Nous nous sommes organisés par groupe de 2 pour traiter les sujets le plus rapidement possible, ce qui nous a donc fait avoir un bon rythme d'avancement, et ainsi d'avoir un robot tout à fait fonctionnel à la date des épreuves évaluatives.

Enfin, vous pouvez retrouver ci-après le sommaire avec les prénoms de ceux ayant rédigé les différentes parties de ce rapport. A noter que le nom de BARROIS Victor n'apparaît nulle part, n'ayant pas rédigé ses parties à temps.

INTRODUCTION → Quentin

- I) Le robot du semestre 2 et ses nouveautés par rapport au S1. → Cost/Corentin
 - 1. Les nouvelles épreuves → Cost
 - 2. Les nouveaux objectifs → Cost
 - 3. La carte hacheur → Corentin

- II) La carte mezzanine → Quentin
 - 1. Comment créer les tensions nécessaires.
 - a. Créer du -5V
 - b. Créer du 3V3
 - 2. Le traitement du signal
 - a. Capter le signal IR
 - b. Supprimer la constante continue
 - c. Amplifier le signal
 - d. Isoler le 5kHz
 - e. Récupérer l'enveloppe du signal
 - f. Rendre le signal binaire
 - 3. Les capteurs ultra-sons
 - 4. L'écran LCD
 - 5. Liste des composants et répartition du travail.

- III) Les compléments → Cost
 - 1. Le module Grove MP3
 - 2. Le module Bluetooth

- IV) Schéma de câblage et commentaire → Quentin

- V) Programmation du robot → Quentin
 - 1. Les encodeurs
 - 2. L'accéléromètre
 - 3. Le carré
 - 4. Le suivi de ligne
 - 5. Eviter un obstacle
 - 6. Détecter la balise et s'en rapprocher

- VI) L'organisation dans l'équipe → Quentin

Conclusion → Quentin

Conclusion

Ce projet SAé robot du S2 BUT1 GEII nous a permis de compléter le robot du semestre 1, en arrivant en fin de semestre 2 avec un robot bien plus performant. Il possède bon nombre de fonctionnalités nouvelles et est donc capable d'effectuer bien plus d'épreuves. Nous sommes donc tous les 4 très satisfaits des capacités actuelles de notre robot.

De plus, durant ce semestre nous avons réellement pu faire le lien entre les différents enseignements du S2, ce qui avait été moins le cas au semestre 1. Cela est appréciable car nous avons directement utilisé nos connaissances des cours d'Elen2 et d'Info2 lors des différentes séances.

Nous avons également bien réussi à gérer nos séances, et nous avons fait un bon travail d'équipe pour obtenir ce résultat, là encore nous pouvons nous en féliciter.

Cependant, un nombre de séances plus important n'aurait pas été de trop. En effet cela nous a obligé à nous dépêcher, et revenir sur notre temps libre afin d'obtenir ces bons résultats avec notre robot.

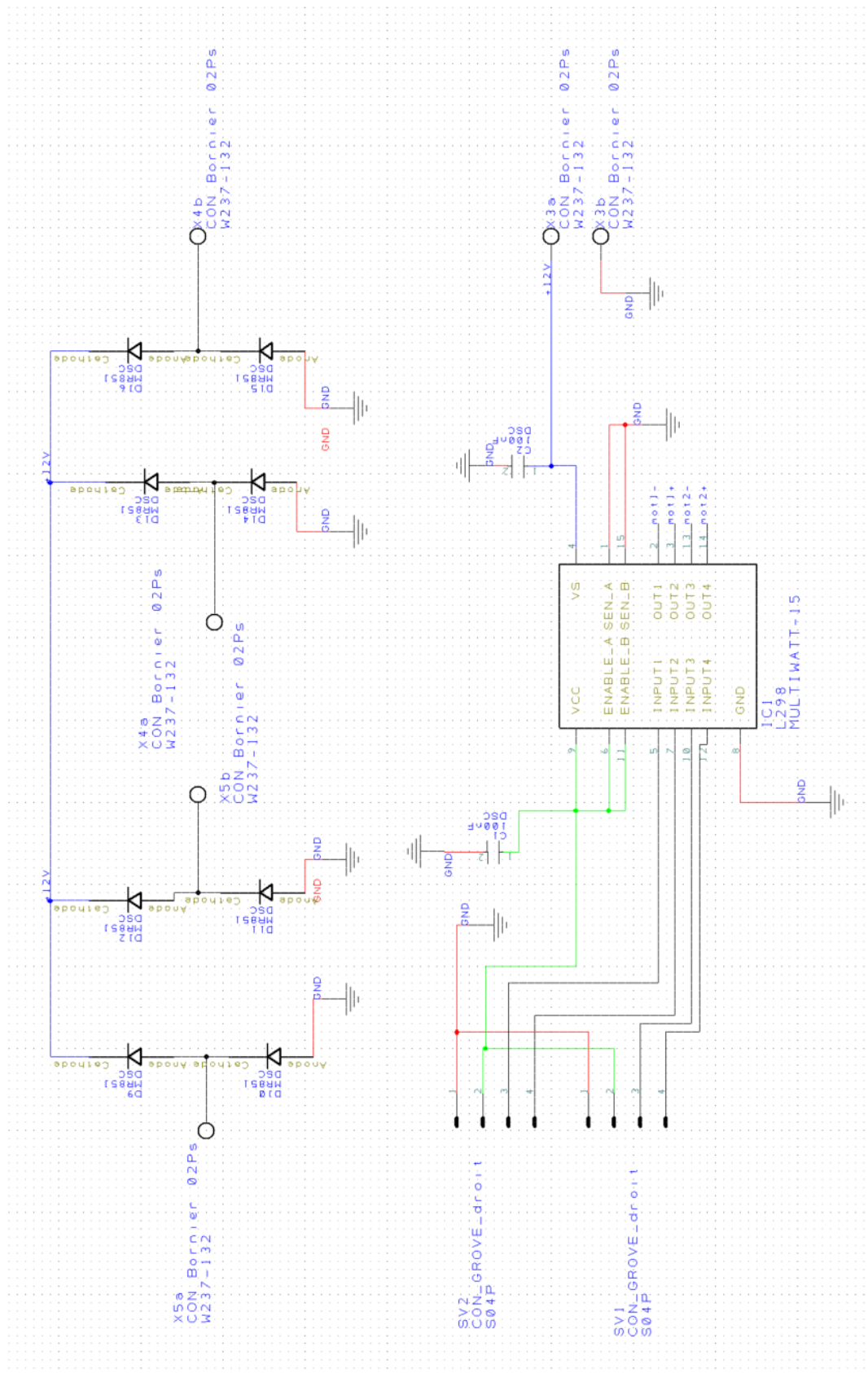
En conclusion, cette SAé du BUT1 est un projet et une expérience très enrichissante, qui nous a permis d'apprendre beaucoup de choses autant sur l'aspect technique que sur l'aspect humain (gestion de projet) et ce, toujours dans une ambiance agréable.

Table des figures

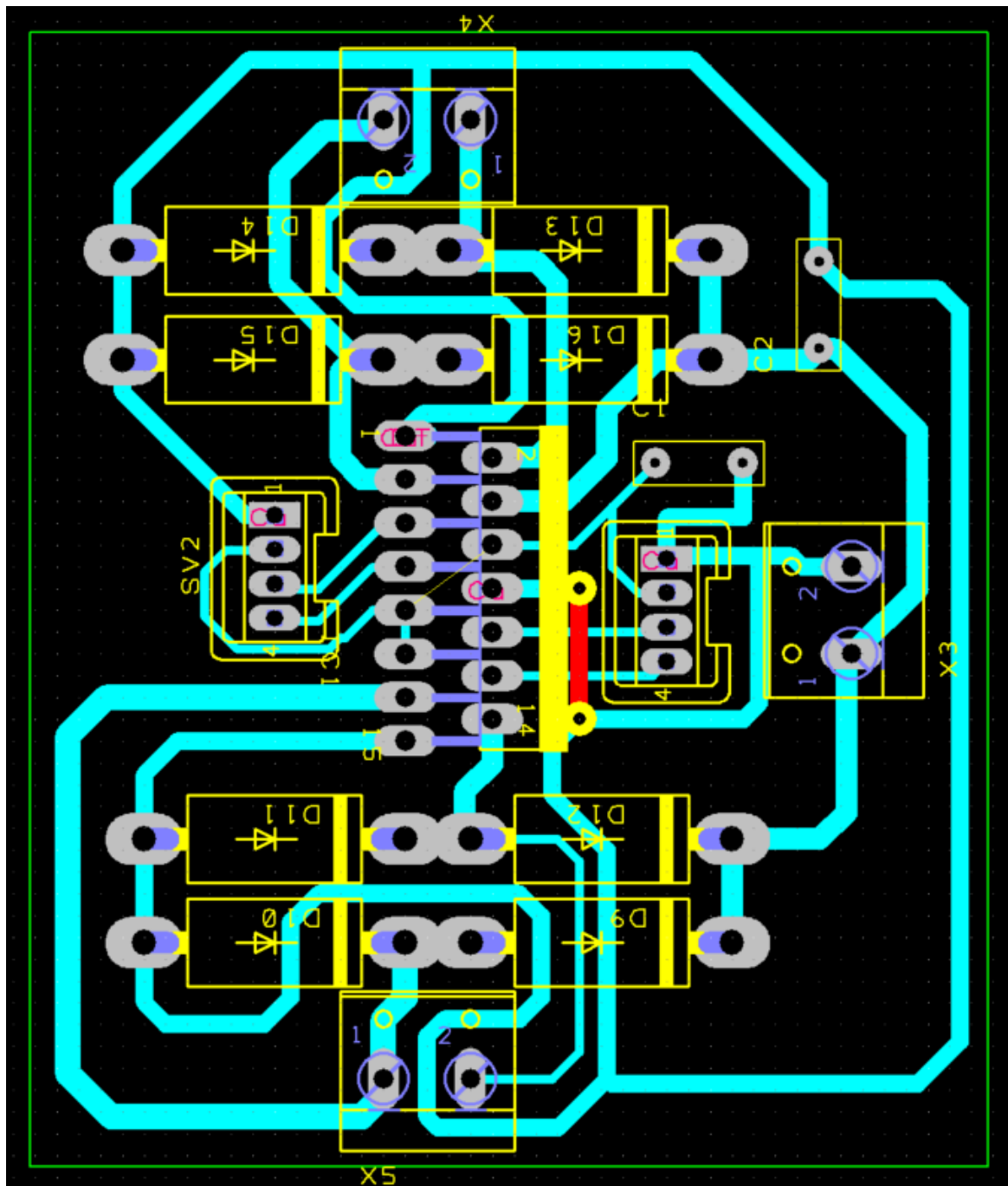
Figure 1 : Pont de Graetz - Hacheur	6
Figure 2 : L298 - Hacheur	6
Figure 3 : PCB - Hacheur	7
Figure 4 : TCL7660 -5V	8
Figure 5 : L78L33 - 3V3.....	9
Figure 6 : Schéma bloc IR	9
Figure 7 : Capter le signal - IR	10
Figure 8 : Filtre passe-bas - IR	10
Figure 9 : Résultat filtre passe-bas - IR	11
Figure 10 : Amplification du signal - IR	12
Figure 11 : Filtre passe-bande - IR	13
Figure 12 : Résultat filtre passe-bande - IR	13
Figure 13 : Détecteur crête imparfait - IR.....	14
Figure 14 : Résultat détecteur crête imparfait - IR	14
Figure 15 : Détecteur crête avec diode sans seuil – IR	15
Figure 16 : LM311 – IR	16
Figure 17 : SRF04	16
Figure 18 : Code 1 - SRF04	17
Figure 19 : Code avec bibliothèque - SRF04	17
Figure 20 : Schematic - SRF04.....	18
Figure 21 : Grove-LCD RGB Backlight.....	18
Figure 22 : Schematic LCD.....	19
Figure 23 : Code LCD - Hello World !.....	19
Figure 24 : Carte Mezzanine – Soudure et PCB.....	21
Figure 25 : Grove MP3.....	22
Figure 26 : Schematic mini carte adaptatrice	23
Figure 27 : PCB mini carte adaptatrice	23
Figure 28 : MP3 - Pause	25
Figure 29 : MP3 – Stop	25
Figure 30 : MP3 – Next	25
Figure 31 : MP3 – Previous	26
Figure 32 : MP3 - Volume control command	26
Figure 33 : MP3 - Secify SD card file name play	26
Figure 34 : HC-05	27
Figure 35 : Schéma de câblage.....	29
Figure 36 : Robot câblé	30
Figure 37 : Encodeurs sur un moteur	31
Figure 38 : Accéléromètre - FRDMKL25Z	33
Figure 39 : Diagramme de Gantt	39

Annexes

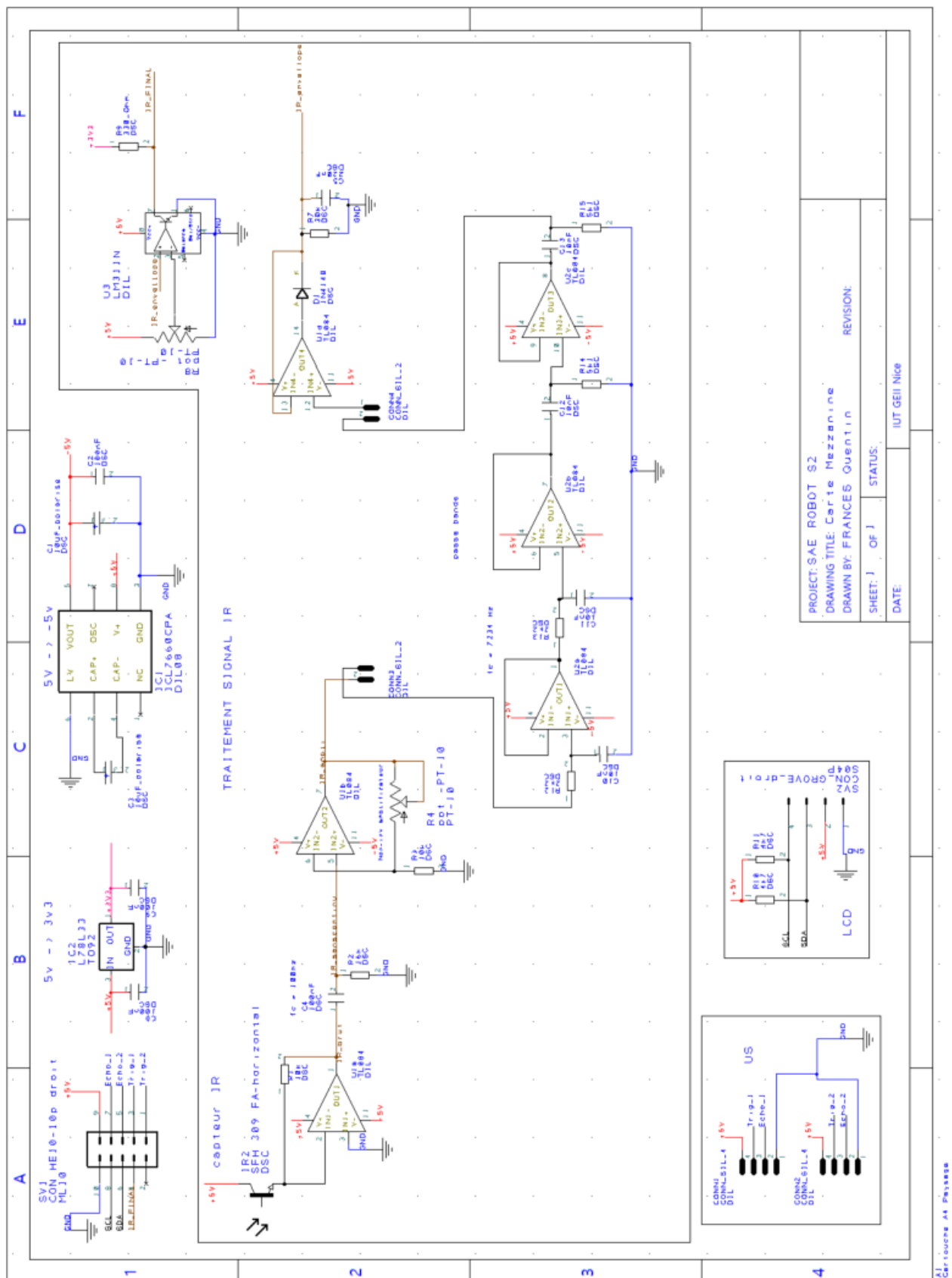
Schematic carte hacheur



PCB carte hacheur :



Schematic carte mezzanine :



PCB carte mezzanine :

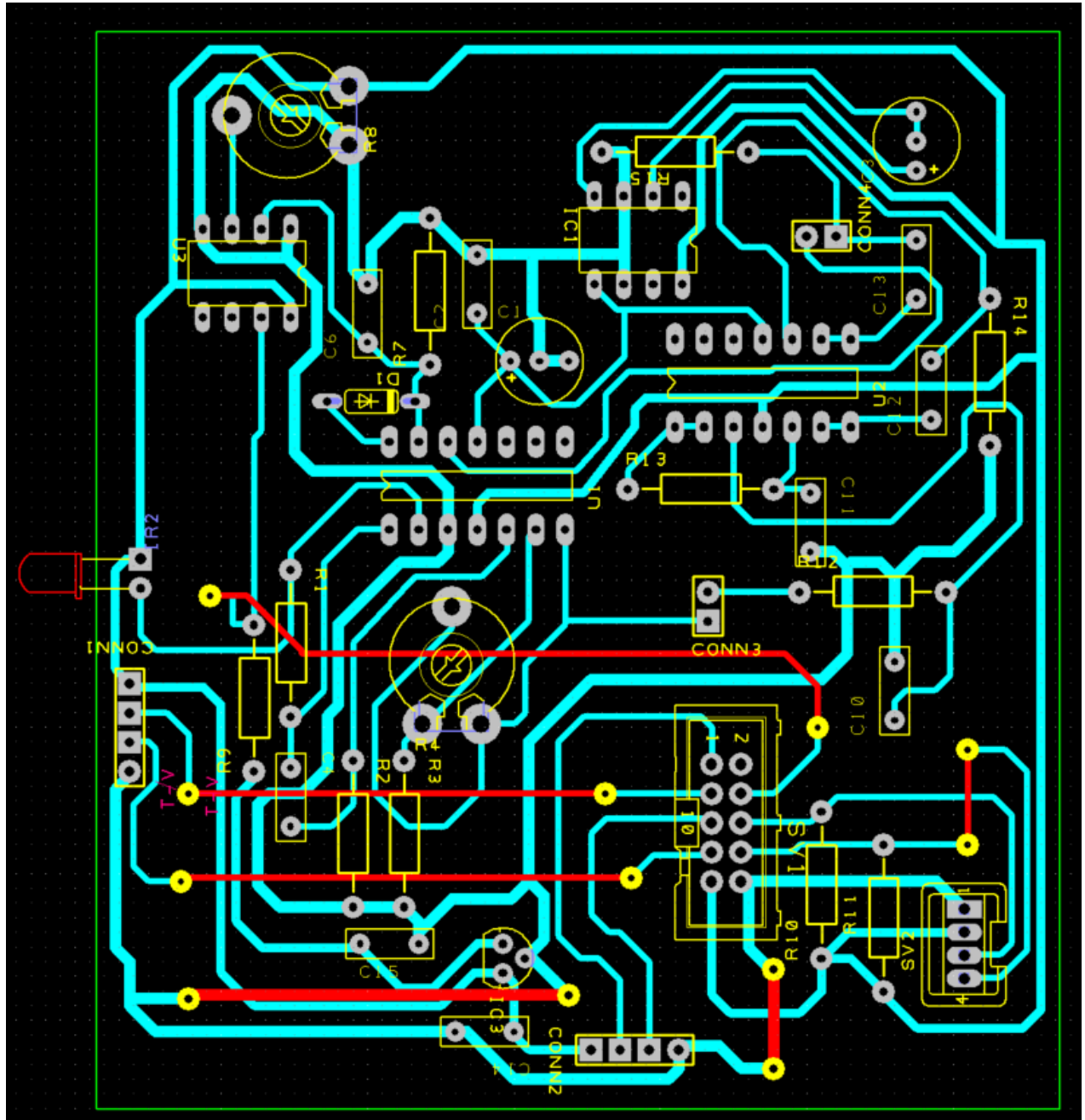


Tableau des commande pour le Grove MP3 :

Communication control command

CMD	Corresponding function	Parameters
A2	Specify SD card root directory index play	File index
A3	Specify SD card file name play	File name
A4	Specify file index play in SD card folder	Folder name, file index
A5	Specify file name play in SD card folder	Folder, file name
A6	Specify U disk root directory index play	File index
A7	Specify U disk file name play	File name
A8	Specify file index play in U disk folder	Folder name, file index
A9	Specify file name play in U disk folder	Folder, file name
AA	Pause command	None
AB	Stop command	None
AC	Next command	None
AD	Previous command	None
AE	Volume control command	Volume levels
AF	Specified play mode	Cycle mode
B0	Combination play	File index
B1	Inter-cut play command	Working drive, file index
B2	Specified EQ mode	EQ mode
BA	Whether need to return code	BA XX
D2	Switch current working drive	working drive

Communication query command

CMD	Corresponding function	Parameters
C1	Query the current volume setting	C1 XX
C2	Query the current operation state	C2 XX
C5	Query the total number of music files in SD card	C5 XXXXXX
C6	Query the total number of music files in specified SD card folder	C6 XXXX
C7	Query the total number of music files in U disk	C7 XXXX
C8	Query the total number of music files in specified U disk folder	C8 XXXX
C9	Query the file track currently playing	C9 XXXX
CA	Query current external connection status	CA XX
CB	Query the song name currently playing	CB XX XX(only support SD card and U disk)

Toutes les fonctions utilisées par la suite :

```

void init()
{
    MOT_G.period_us(100);
    MOT_D.period_us(100);
    am1.rise(&incrementation1);
    am2.rise(&incrementation2);
}

void incrementation1(void)
{
    if(bm1.read() == 0)
        cpt1++;
    else
        cpt1--;
}

void incrementation2(void)
{
    if(bm2.read() == 1)
        cpt2++;
    else
        cpt2--;
}

void vitesse(float a, float b) {
    //batterie = ((bat.read() * 3.3) * (59.0 / 12.0)) + 0.6;
    if(a >= 0)
    {
        SMOT_G.write(0);
        MOT_G.pulsewidth_us(a);
    }
    else {
        SMOT_G.write(1);
        MOT_G.pulsewidth_us(100-(-1 * a));
    }
    if(b >= 0)
    {
        SMOT_D.write(0);
        MOT_D.pulsewidth_us(b);
    }
    else {
        SMOT_D.write(1);
        MOT_D.pulsewidth_us(100-(-1 * b));
    }
}

89 void tourne(int16_t angle)
90 {
91     cpt1 = 0;
92     cpt2 = 0;
93     angle = (angle*465/360)/2;
94
95     do{
96         if(cpt1 > (-1 * cpt2))
97             vitesse(35.0,-40.0);
98         else
99             vitesse(35.0,-30.0);
100         wait_us(150.0);
101     }while(cpt1 < angle);
102 }
103
104 void avance(int16_t dist)
105 {
106     cpt1 = 0;
107     cpt2 = 0;
108     do{
109         if(cpt1>cpt2)
110         {
111             vitesse(35.0,40.0);
112         }
113         else if(cpt1<=cpt2)
114         {
115             vitesse(35.0,30.0);
116         }
117         wait_us(150);
118     }while(cpt1 < dist);
119 }

```

Code accéléromètre :

```

1  #include "mbed.h"
2
3  #define REG_WHO_AM_I    0x00 //Numéro du registre à consulter pour connaître l'identifiant du composant
4  #define REG_CTRL_REG_1  0x2A //Numéro du registre à remplir avec 0x01 pour activer le composant
5  #define REG_OUT_X_MSB    0x01 //Numéro du registre à consulter pour acquérir l'accélération en X sur 2 octets
6  #define REG_OUT_Y_MSB    0x03 //Numéro du registre à consulter pour acquérir l'accélération en Y sur 2 octets
7  #define REG_OUT_Z_MSB    0x05 //Numéro du registre à consulter pour acquérir l'accélération en Z sur 2 octets
8  #define UINT14_MAX       16384 //correspond au nombre décimal max codé sur 14 bits
9  #define adresse_I2C      0x3A //Adresse 8 bits du composant MMA8451 sur le bus I2C
10
11  I2C composant( PTE25 , PTE24 ); //Création de l'objet I2C appelé "composant" connecté aux broches PTE25 (SDA) et PTE24 (SCL)
12  InterruptIn am1(D2); // encodeur mot_G
13  InterruptIn am2(D11); // encodeur mot_D
14  DigitalIn bm1(D3); // encodeur mot_G
15  DigitalIn bm2(D10); // encodeur mot_D
16  PwmOut MOT_D(D8); // puissance mot_D
17  PwmOut MOT_G(D6); // puissance mot_G
18  DigitalOut SMOT_D(D9); // sens mot_D
19  DigitalOut SMOT_G(D7); // sens mot_G
20  int16_t cpt1 = 0, cpt2 = 0;
21
22  void init();
23  void tourne(int16_t);
24  void avance(int16_t);
25  void incrementation1(void);
26  void incrementation2(void);
27  void vitesse(float,float); //Fonction d'initialisation
28  //char getWhoAmI() ; //Fonction permettant de récupérer l'identifiant du composant (ID)
29  int getAccAxis(char AXE) ; //Fonction permettant de récupérer la valeur de l'accélération suivant tel ou tel AXE
30
31  int main() { //Programme principal
32      int val_z, etat = 0; float val_z_g; //variable qui va stocker la mesure de l'accélération
33      init(); //Mise en marche du circuit
34      //printf("je suis = %d\n",getWhoAmI()); //Affichage de l'ID
35      while(1) { //Boucle infinie
36          val_z=getAccAxis(REG_OUT_Z_MSB); // Lecture de l'acceleration en Z - valeur brute
37          val_z_g = (float)val_z / 4000;
38          printf("etat : %d, val z : %lg\n", etat, val_z_g);
39          switch(etat){
40              case 0 :
41                  vitesse(0.0,0.0);
42                  if(val_z_g > 1.0){
43                      etat = 1;
44                  }
45                  break;
46              case 1 :
47                  avance(400);
48                  etat=2;
49                  break;
50              case 2:
51                  vitesse(0.0,0.0);
52                  wait(5);
53                  etat = 0;
54                  break;
55          }

```

```
56     wait(0.1);
57     }
58 }
59
60 void init() { //Fonction d'initialisation
61     char data[2] = {REG_CTRL_REG_1, 0x01}; //On remplit un tableau de char avec 2 valeurs : numéro de registre et valeur
62     composant.write(adresse_I2C, data, 2); //Activation du composant MMA8451
63     MOT_G.period_us(100);
64     MOT_D.period_us(100);
65     am1.rise(&incrementation1);
66     am2.rise(&incrementation2);
67 }
68
69 /*char getWhoAmI() { //Fonction permettant de récupérer l'identifiant du composant (ID)
70     char registre[1]={REG_WHO_AM_I}; //Création et initialisation d'un tableau de char appelé "registre"
71     char qui_je_suis[1]; //Création d'un tableau de 1 char qui permettra de stocker l'identifiant (ID)
72     composant.write(adresse_I2C, registre, 1, true); //Ecriture sur le bus = "Esclave n°0x3A, donne moi ton identifiant" - true = "j'attends ta réponse"
73     composant.read(adresse_I2C, qui_je_suis, 1); //Attente de réception de 1 octet : l'ID
74     return qui_je_suis[0]; //La fonction renvoie l'ID au programme principal
75 }*/
76
77 int getAccAxis(char AXE) { //Fonction permettant de récupérer la valeur de l'accélération suivant tel ou tel AXE
78     int acceleration; //variable qui va stocker la mesure de l'accélération
79     char registre[1]={AXE}; //Création d'un tableau de 1 char stockant le n° du registre à interroger (AXE passé en paramètre)
80     char data[2]; //Création d'un tableau de 2 char qui stockera la réponse du composant
81     composant.write(adresse_I2C, registre, 1, true); //Ecriture sur le bus = "Esclave n°0x3A, donne moi l'accélération suivant AXE" - true = "j'attends ta réponse"
82     composant.read(adresse_I2C, data, 2); //Attente de réception de 2 octets : données de l'accélération
83     acceleration = (data[0] << 6) | (data[1] >> 2); //Traitement des données pour reconstituer la valeur de l'accélération
84     if (acceleration > UINT14_MAX/2) //Si la valeur dépasse la moitié de 2^14 - 1
85         acceleration = acceleration - UINT14_MAX; //Elle est en réalité négative, on lui soustrait 2^14 - 1
86     return acceleration; //La fonction renvoie la valeur de l'accélération - 4096 <=> acceleration de +1 g [m.s^(-2)]
87 }
```

Suivi de ligne :

```
1  #include "Grove_LCD_RGB_Backlight.h"
2  #include "SRF05.h"
3  #include "mbed.h"
4  #define VMOY 30
5  #define K 30
6
7  Grove_LCD_RGB_Backlight LCD(D14, D15);
8  AnalogIn capt_1(A1);
9  AnalogIn capt_2(A2);
10 AnalogIn capt_3(A3);
11 AnalogIn capt_4(A4);
12 SRF05 Capt_avant(A5, D4);
13 SRF05 Capt_gauche(PTE30, D5);
14 DigitalIn Capteur_IR(D13);
15 InterruptIn am1(D2); // encodeur mot_G
16 InterruptIn am2(D11); // encodeur mot_D
17 DigitalIn bm1(D3); // encodeur mot_G
18 DigitalIn bm2(D10); // encodeur mot_D
19 PwmOut MOT_D(D8); // puissance mot_D
20 PwmOut MOT_G(D6); // puissance mot_G
21 DigitalOut SMOT_D(D9); // sens mot_D
22 DigitalOut SMOT_G(D7); // sens mot_G
23 // DigitalIn JACK(PTE20); // enfoncé à 0;
24 DigitalIn FDC(PTE21); // enfoncé à 0;
25 DigitalOut myled(LED1);
26
27 Timer t;
28
29 int16_t cpt1 = 0, cpt2 = 0, IR, cpt = 0, cp = 0, jack,
30         fdc;
31
32 char tab[20];
33
34 void init();
35 void tourne(int16_t);
36 void avance(int16_t);
37 void incrementation1(void);
38 void incrementation2(void);
39 void vitesse(float, float);
40 void toutdroit();
41
42 int main() {
43     int8_t etat = 0;
44     int16_t cpt1av;
45     float captav, captg, captavav, captgav;
46     float capt1, capt2, capt3, capt4, err, cmdG, cmdD, delta;
47
48     //JACK.mode(PullUp);
49     FDC.mode(PullUp);
50     init();
```

```
51
52     while (1) {
53         fdc = FDC.read();
54         captav = Capt_avant.read();
55         captg = Capt_gauche.read();
56         capt2 = capt_2.read();
57         capt3 = capt_3.read();
58         delta = capt2 - capt3;
59
60
61         if(delta<=0) {
62             SMOT_G.write(0);
63             SMOT_D.write(0);
64             MOT_G.pulsewidth_us(50.0);
65             MOT_D.pulsewidth_us(30.0*delta);
66         }
67         if(delta>=0) {
68             SMOT_G.write(0);
69             SMOT_D.write(0);
70             MOT_G.pulsewidth_us(30.0*delta);
71             MOT_D.pulsewidth_us(50.0);
72         }
73         else if(fdc == 0) {
74             vitesse (0.0, 0.0);
75             LCD.setRGB(255,255,255);
76             LCD.locate(6,0);
77             sprintf(tab,"WIN");
78             LCD.print(tab);
79         }
```

Contourner un obstacle :

```
1  #include "Grove_LCD_RGB_Backlight.h"
2  #include "SRF05.h"
3  #include "mbed.h"
4  #define VMOY 30
5  #define K 30
6
7  Grove_LCD_RGB_Backlight LCD(D14, D15);
8  AnalogIn capt_1(A1);
9  AnalogIn capt_2(A2);
10 AnalogIn capt_3(A3);
11 AnalogIn capt_4(A4);
12 SRF05 Capt_avant(A5, D4);
13 SRF05 Capt_gauche(PTE30, D5);
14 DigitalIn Capteur_IR(D13);
15 InterruptIn am1(D2); // encodeur mot_G
16 InterruptIn am2(D11); // encodeur mot_D
17 DigitalIn bm1(D3); // encodeur mot_G
18 DigitalIn bm2(D10); // encodeur mot_D
19 PwmOut MOT_D(D8); // puissance mot_D
20 PwmOut MOT_G(D6); // puissance mot_G
21 DigitalOut SMOT_D(D9); // sens mot_D
22 DigitalOut SMOT_G(D7); // sens mot_G
23 // DigitalIn JACK(PTE20); // enfoncé à 0;
24 DigitalIn FDC(PTE21); // enfoncé à 0;
25 DigitalOut myled(LED1);
26
27 Timer t;
28
29 int16_t cpt1 = 0, cpt2 = 0, IR, cpt = 0, cp = 0, jack,
30         fdc;
31
32 char tab[20];
33
34 void init();
35 void tourne(int16_t);
36 void avance(int16_t);
37 void incrementation1(void);
38 void incrementation2(void);
39 void vitesse(float, float);
40 void toutdroit();
41
42 int main() {
43     int8_t etat = 0;
44     int16_t cpt1av;
45     float captav, captg, captavav, captgav;
46     float capt1, capt2, capt3, capt4, err, cmdG, cmdD;
47
48     //JACK.mode(PullUp);
49     FDC.mode(PullUp);
50     init();
51 }
```

```
52     while (1) {
53         captav = Capt_avant.read();
54         captg = Capt_gauche.read();
55         capt2 = capt_2.read();
56         capt3 = capt_3.read();
57         switch (etat) {
58             case 0: if (captav <= 15 && captav > 15) {
59                 vitesse(0.0, 0.0);
60                 wait(1);
61                 tourne(90);
62                 wait(1);
63                 cpt1 = 0;
64                 cpt2 = 0;
65                 etat = 1;
66             }
67             break;
68             case 1:
69                 if (captg >= 40 && captg < 40) {
70                     cpt1av = cpt1;
71                     avance(100);
72                     wait(1);
73                     tourne(300);
74                     wait(1);
75                     avance(200);
76                     wait(1);
77                     tourne(300);
78                     wait(1);
79                     avance(cpt1av+100);
80                     wait(1);
81                     tourne(90);
82                     wait(1);
83                     cpt1 = 0;
84                     cpt2 = 0;
85                     etat = 0;
86                 }
87                 break;
88         }
89     }
90     switch (etat) {
91     case 0:
92         toutdroit();
93         break;
94     case 1: toutdroit(); break;
95     }
96     sprintf(tab,"capteurav = %.2f",captav);
97     LCD.locate(0,0);
98     LCD.print(tab);
99     sprintf(tab,"capteurg = %.2f",captg);
100    LCD.locate(0,1);
101    LCD.print(tab);
```


Code balise :

```
#include "mbed.h"

DigitalIn Capteur_IR(D13);
InterruptIn am1(D2); // encodeur mot_G
InterruptIn am2(D11); // encodeur mot_D
DigitalIn bm1(D3); // encodeur mot_G
DigitalIn bm2(D10); // encodeur mot_D
PwmOut MOT_D(D8); // puissance mot_D
PwmOut MOT_G(D6); // puissance mot_G
DigitalOut SMOT_D(D9); // sens mot_D
DigitalOut SMOT_G(D7); // sens mot_G
DigitalIn JACK(PTE20); // enfoncé à 0;
DigitalIn FDC(PTE21); // enfoncé à 0;
DigitalOut myled(LED1);
int16_t cpt1 = 0, cpt2 = 0, IR,cpt=0,cp=0;

void init();
void tourne(int16_t);
void avance(int16_t);
void incrementation1(void);
void incrementation2(void);
void vitesse(float,float);
void avance();
int main() {

    JACK.mode(PullUp);
    FDC.mode(PullUp);
    init();

    while(1)
    {
        IR = Capteur_IR.read();
        myled.write(1-IR);
        if(IR ==1) {
            vitesse(35.0,35.0);
        }
        else{
            vitesse(-30.0,30.0);
        }
    }
}
```